

CoDeSys V2.3



CoDeSys

Sua Licença para o Sucesso

CoDeSys V2.3

Controller Development System

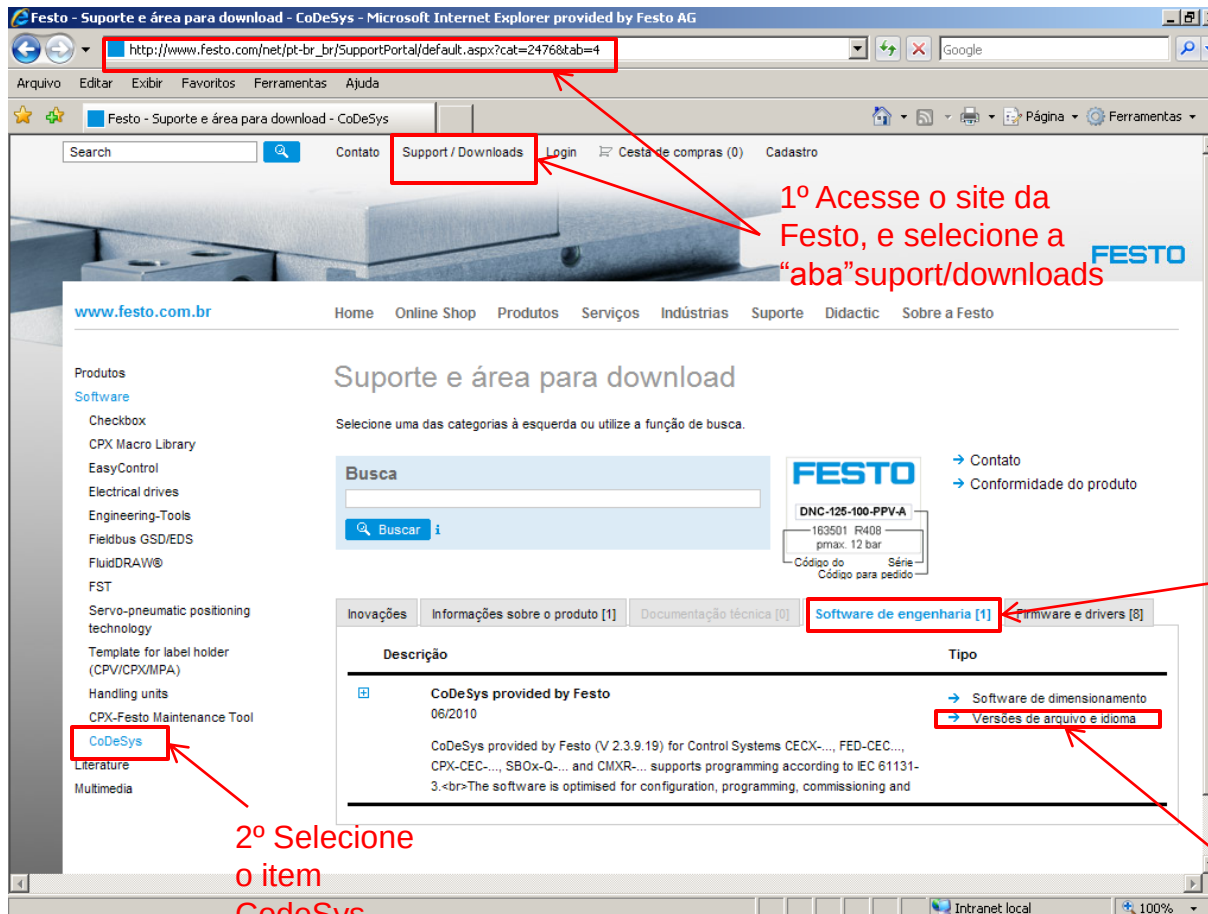
O que é o CoDeSys?

- Software de Programação;
- Ferramenta de Teste;
- Ferramenta de Debug;
- Simulador;
- Ferramenta de documentação;
- Supervisão;



CoDeSys é um ambiente completo de desenvolvimento

CoDeSys V2.3 - Download



1º Acesse o site da Festo, e selecione a "aba" support/downloads

2º Selecione o item CoDeSys

3º Selecione a "aba" software de engenharia

4º clique no item para realizar o download

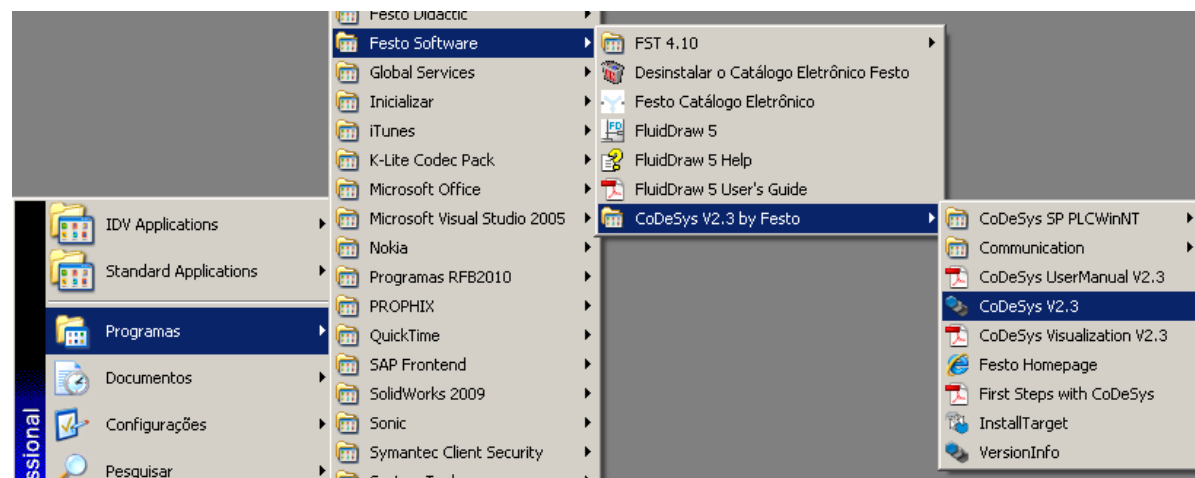
- É possível realizar o download "free" da versão 2.3 completa através do site da Festo

CoDeSys V2.3 - Instalação

- **Requerimentos para instalação:**
 - Windows 2000 SP4 (Service Pack 4)
 - Windows XP SP2 (Service Pack 2)
 - Windows Vista
 - Windows 7 (Professional)
- **Selecione o arquivo CodeSys_pbf_v2.393.e**



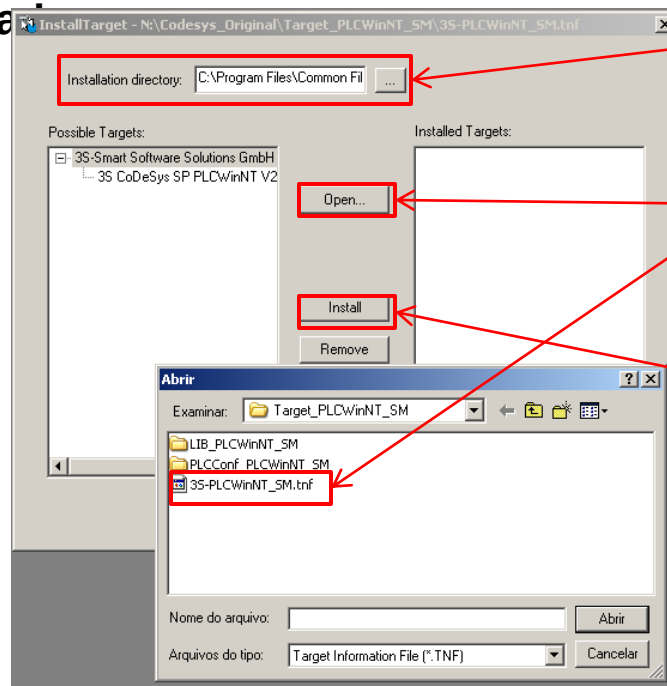
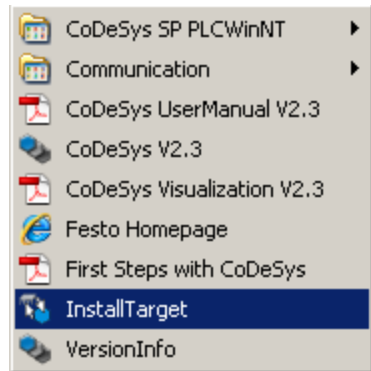
- **Depois de instalado, o CodeSys pode ser encontrado conforme o caminho mostrado na figura ao lado.**



CoDeSys V2.3 – Targets

- Targets** são aplicativos específicos que devem ser instalados de acordo com o **tipo do controlador ou driver** que deseja ser programado. Quando instalado, o mesmo **disponibiliza** no codesys um conjunto de **ferramentas e bibliotecas** que correspondem ao dispositivo desejado.

Para Instalar o Target desejado



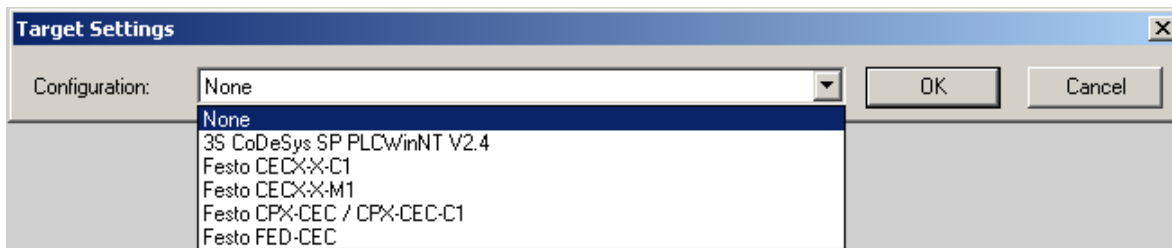
1º Diretório de instalação

2º Abra o arquivo com extensão (.tnf) do driver ou controlador desejado.

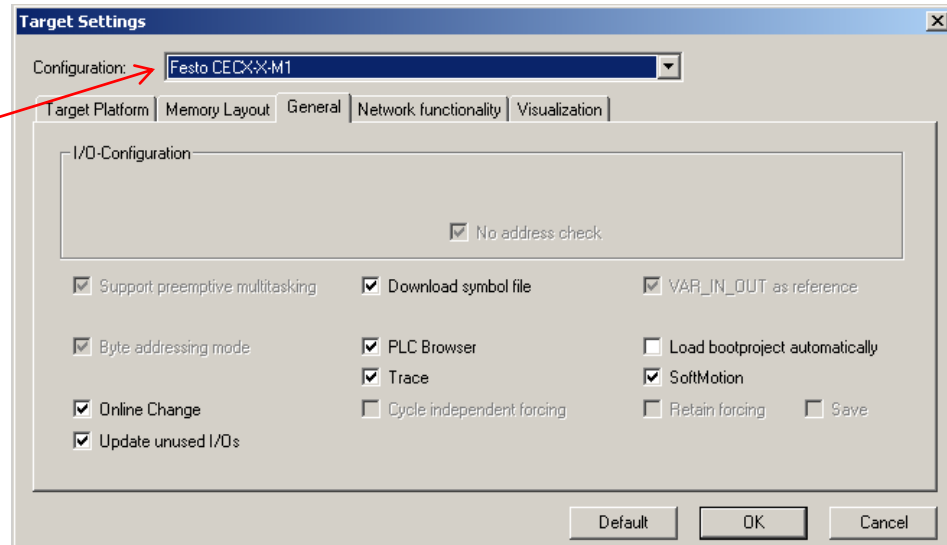
3º Selecione o Target e clique em Install

CoDeSys V2.3 - Criando um novo projeto

Selecione : File → New → selecione o Target desejado.



Cada dispositivo possui um conjunto de opções que devem ser definidas para dar início ao projeto.

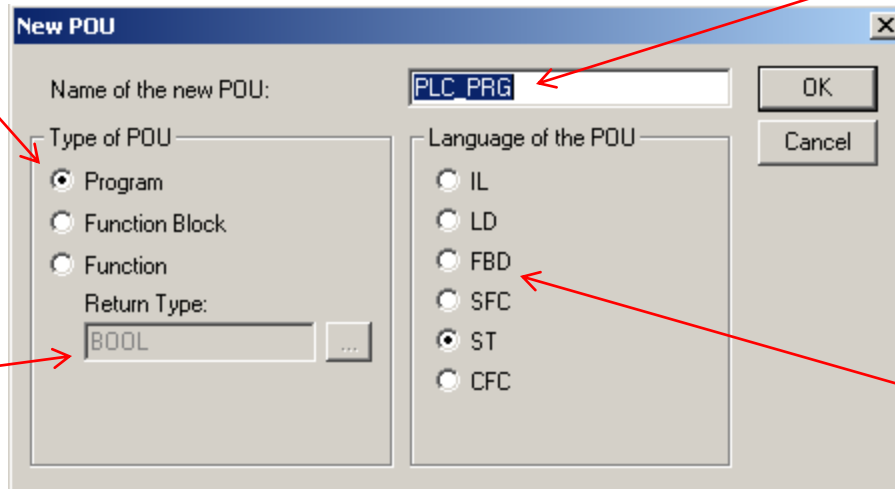


CoDeSys V2.3 - Criando um novo projeto

POU = Program Organization Unit;

Tipo de programa

Tipo da variável
de retorno,
quando o
programa é do
tipo função



Nome do
programa

Linguagem de
programação

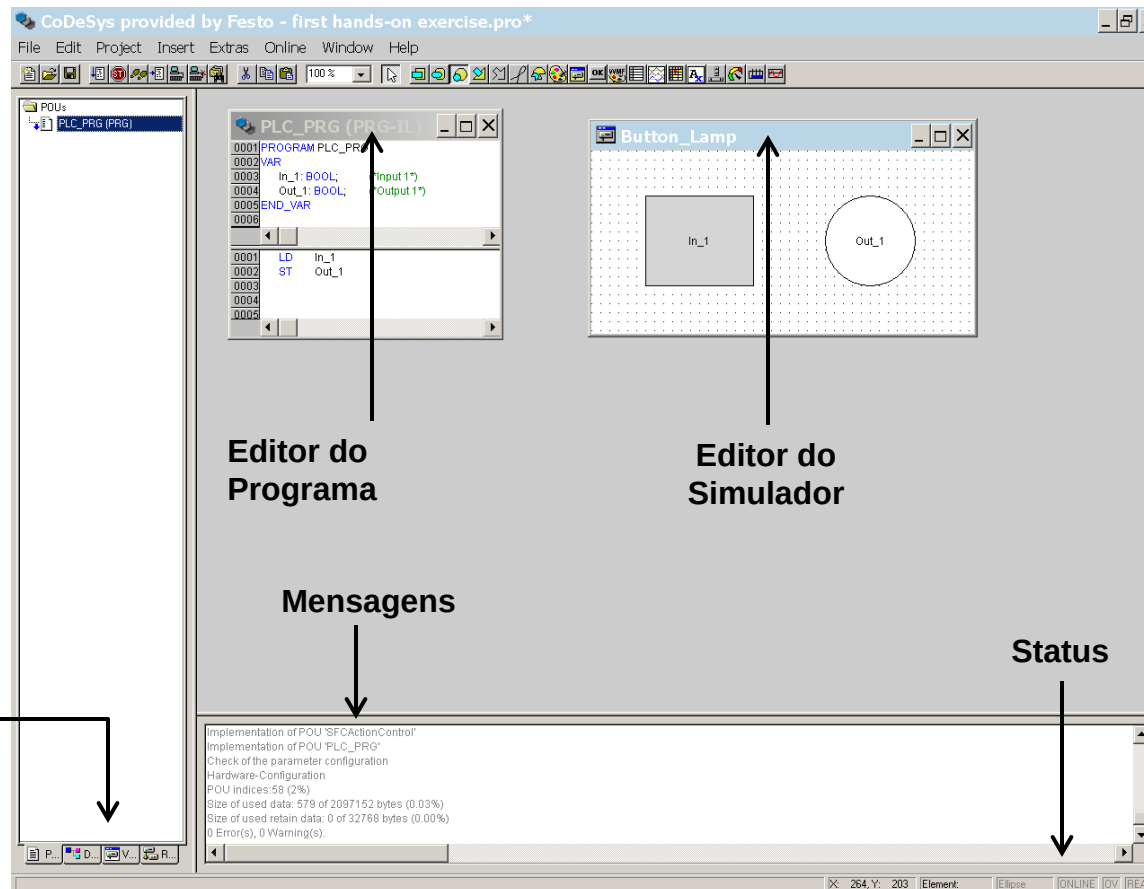
CoDeSys V2.3 - Projeto

Constituição do Projeto no CoDeSys

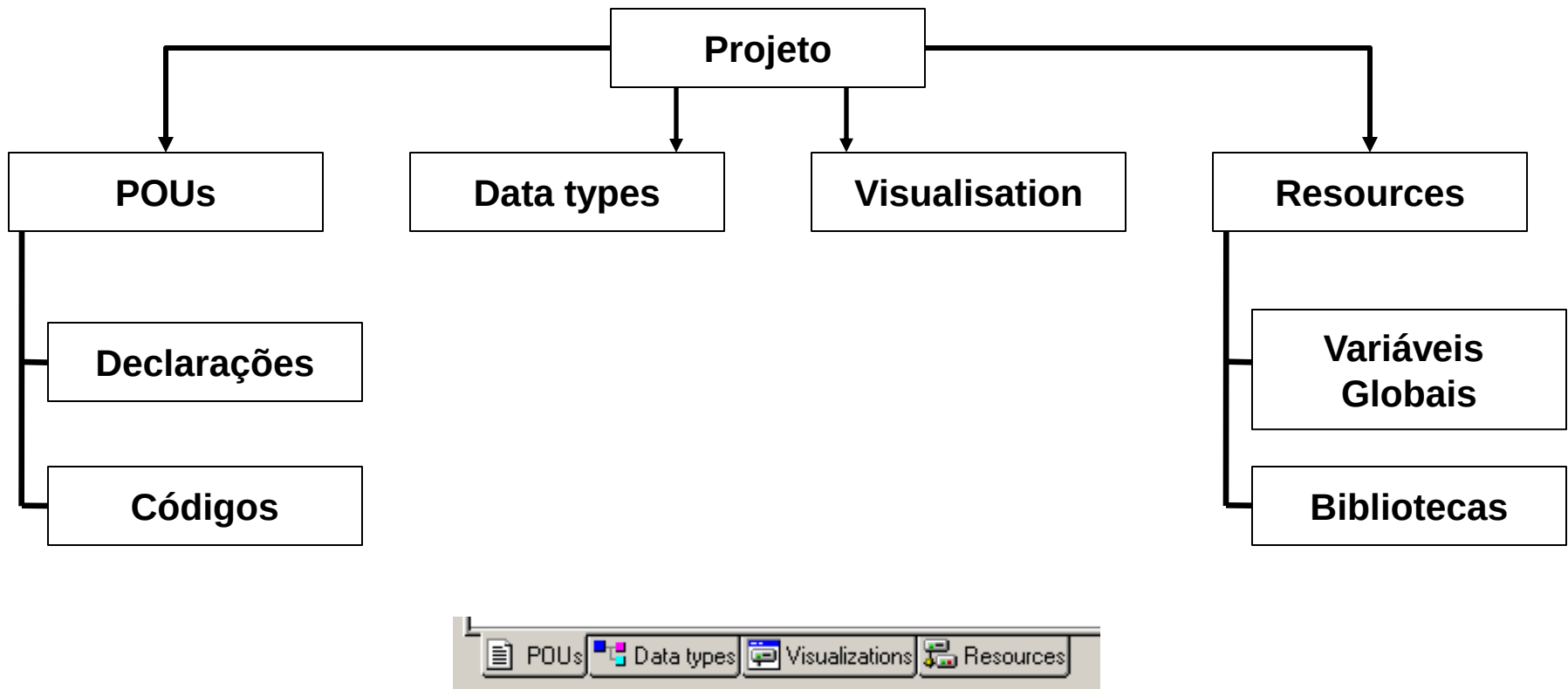
- Salvo em um único arquivo de extensão “**.pro**”
- Formado por uma unidade organizacional de programa (**P**rogram **O**rganization **U**nit), ou seja, funções, blocos funcionais e programas;
- O projeto sempre deve incluir um programa chamado **PLC_PRG** (programa executado ciclicamente)

CoDeSys fundamentals

Projeto
Aberto

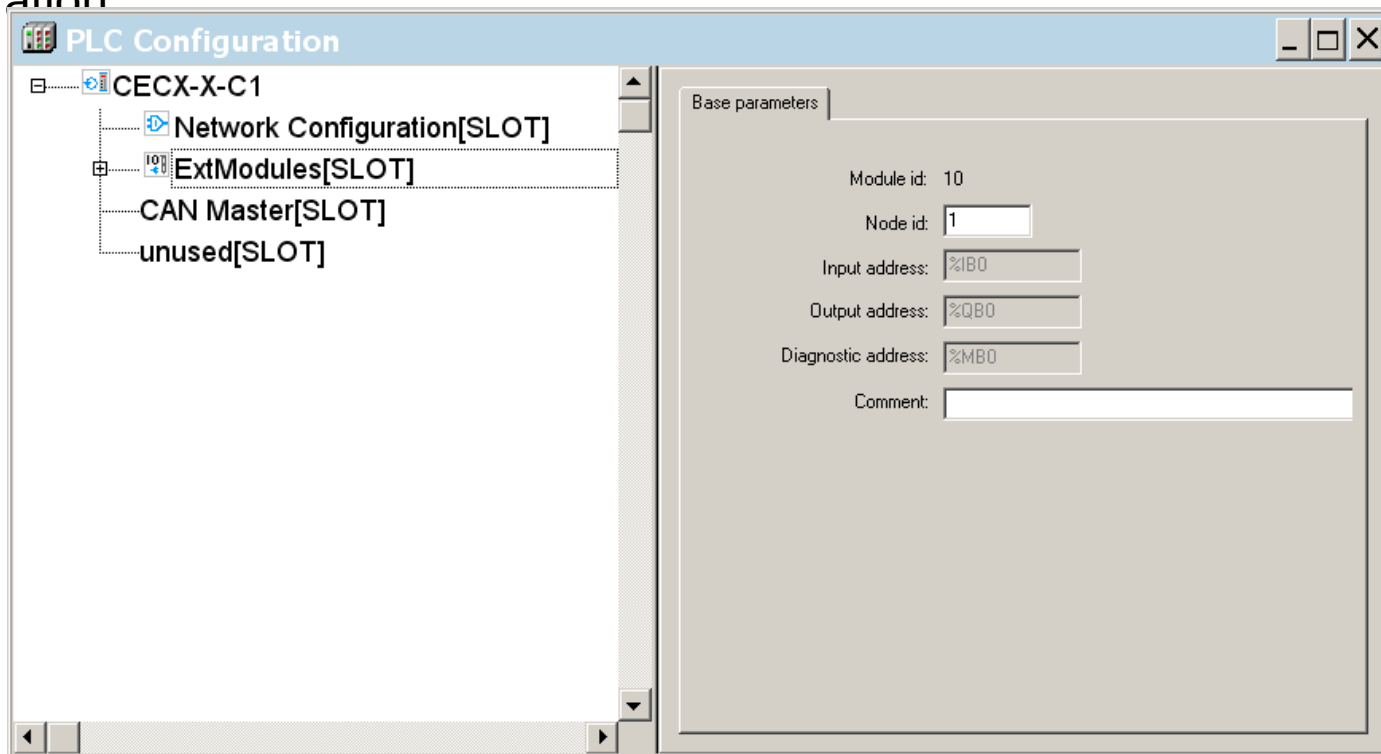


CoDeSys V2.3 - Estrutura de um Projeto



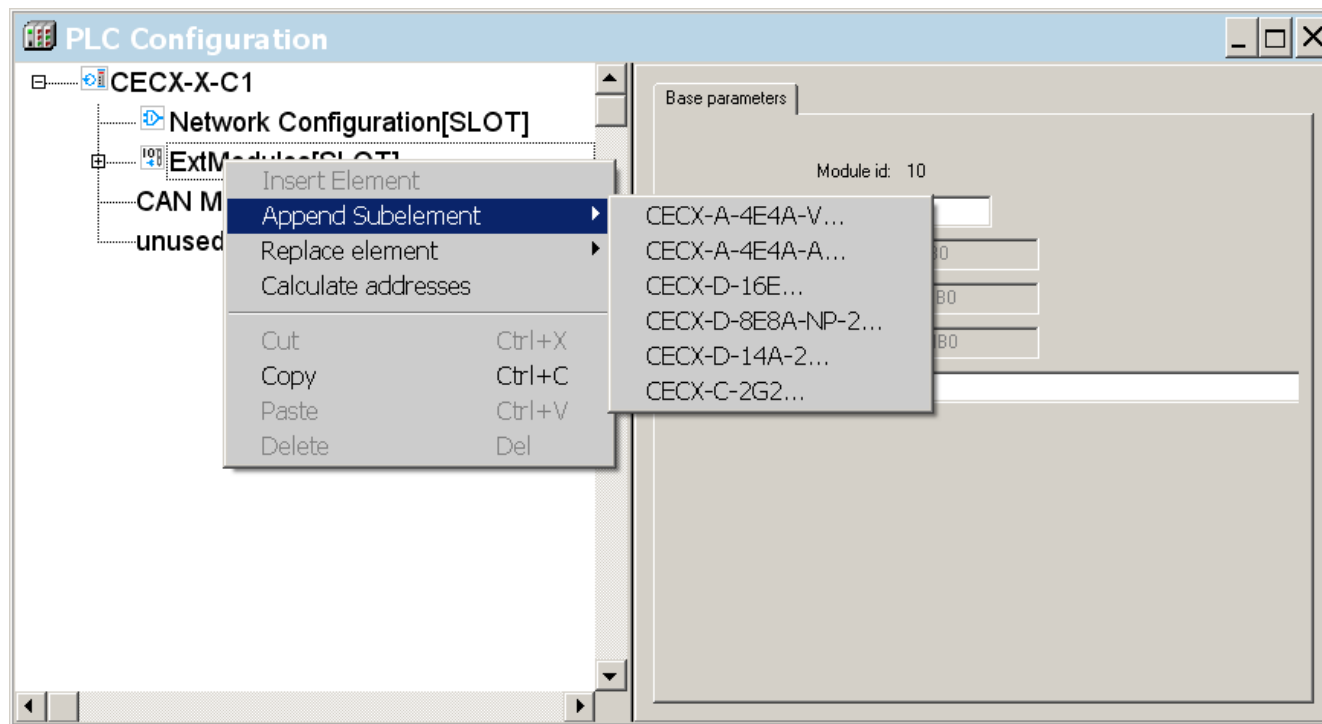
PLC Configuration – Configuração de Hardware

Após criar um novo projeto e selecionar o controlador desejado, é possível configurar os módulos de entrada e saída e comunicação na aba Resource->PLC Configuration



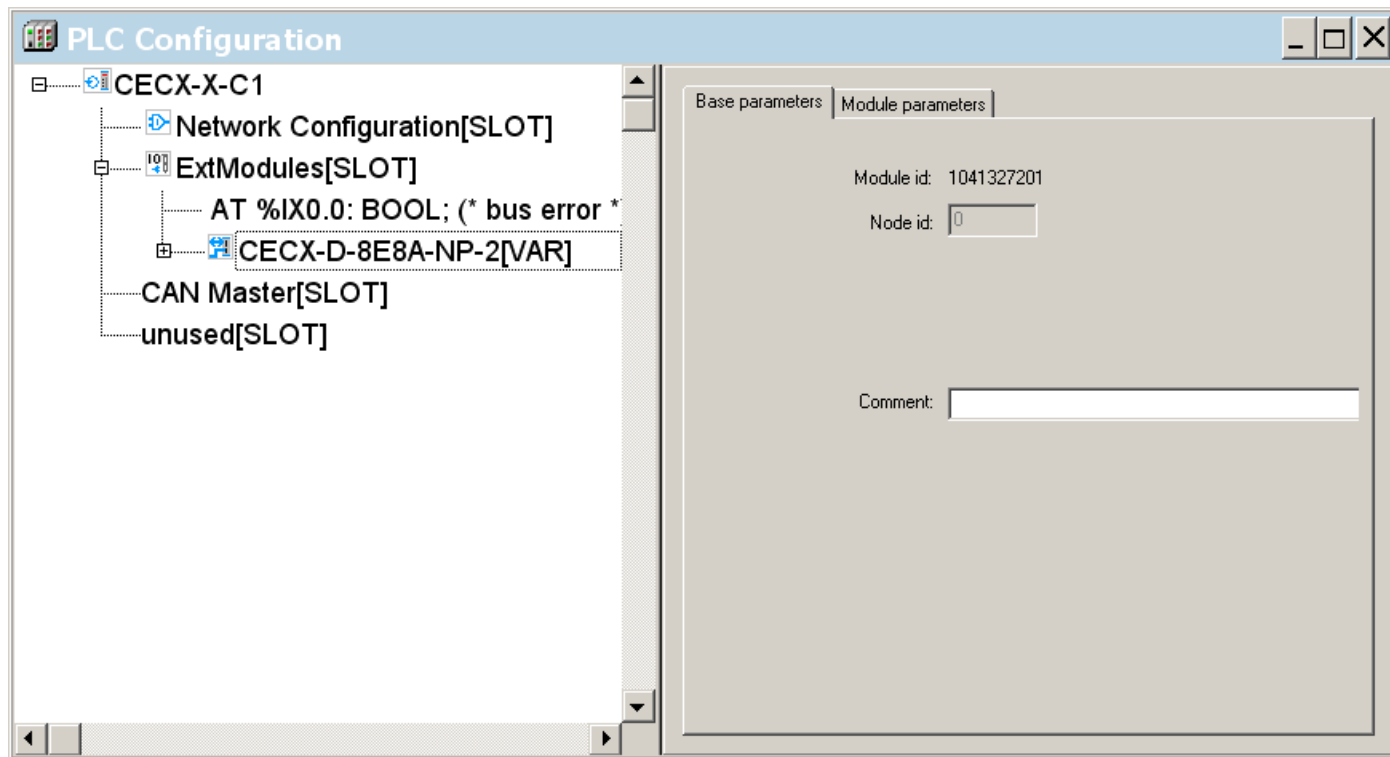
PLC Configuration – Configuração de Hardware

- Clique no slot **ExtModules**, e então, clique com o botão direito do mouse;
- Selecione **Append Subelement** e selecione o módulo desejado;
- Neste exemplo, será selecionado o módulo CECX-8E8A-NP-2



PLC Configuration – Configuração de Hardware

Módulo CECX-D-8E8A-NP-2[VAR] adicionado

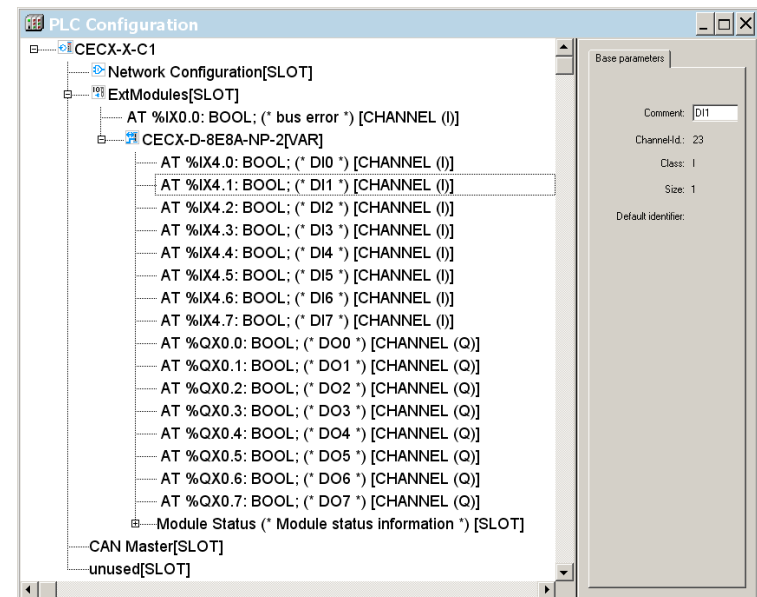


Nomeando Entradas e Saídas

- AT %IX4.0; BOOL; (*DI0*) [CHANNEL(I)]
- AT %QX4.0; BOOL; (*DO0*) [CHANNEL(Q)]
- Clicando em **A** de **AT**, um campo é aberto permitindo adicionar um identificador para o endereço/bit;

Exemplo:

- **Liga** AT %IX4.0; BOOL; (*DI0*) [CHANNEL(I)]
- **V1_1Y1** AT %QX4.0; BOOL; (*DO0*) [CHANNEL(Q)]



Endereçamento Absoluto

Sinal de Porcentagem ‘%’

Prefixo

I Input

Q Output

M Marker

Tipo de Dado

X: **bit**

B: **Byte (8 Bits)**

W: **Word (16 Bits)**

D: **Douple Word (32 Bits)**

Exemplos:

%IW215

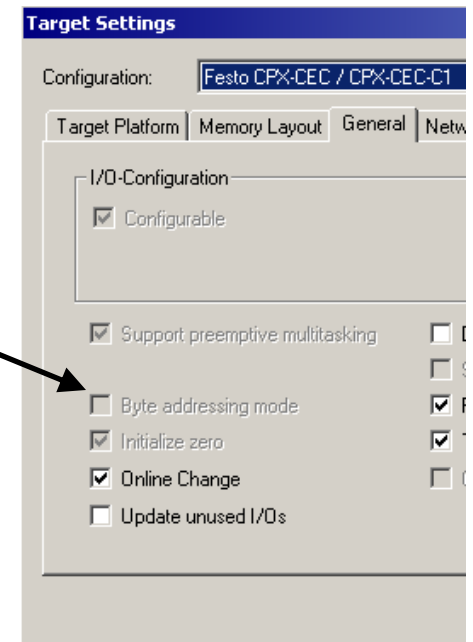
%QX1.1

%MD48

Formas de Endereçamento: Word

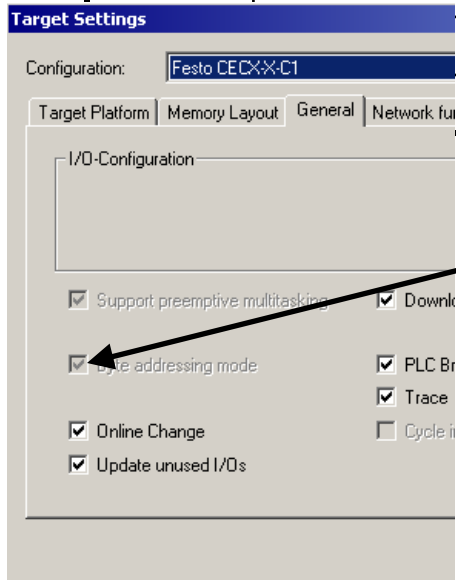
%IX0.0- %IX0.7	%IX0.8- %IX0.15	%IX1.0- %IX1.7	%IX1.8- %IX1.15	%IX2.0- %IX2.7	%IX2.8- %IX2.15	%IX3.0- %IX3.7	%IX3.8- %IX3.15	%IX4.0- %IX4.7	%IX4.8- %IX4.15	%IX5.0- %IX5.7	%IX5.8- %IX5.15
%IB0	%IB1	%IB2	%IB3	%IB4	%IB5	%IB6	%IB7	%IB8	%IB9	%IB10	%IB11
%IW0		%IW1		%IW2		%IW3		%IW4		%IW5	
%ID0				%ID1				%ID2			

**Endereçamento por
Word**



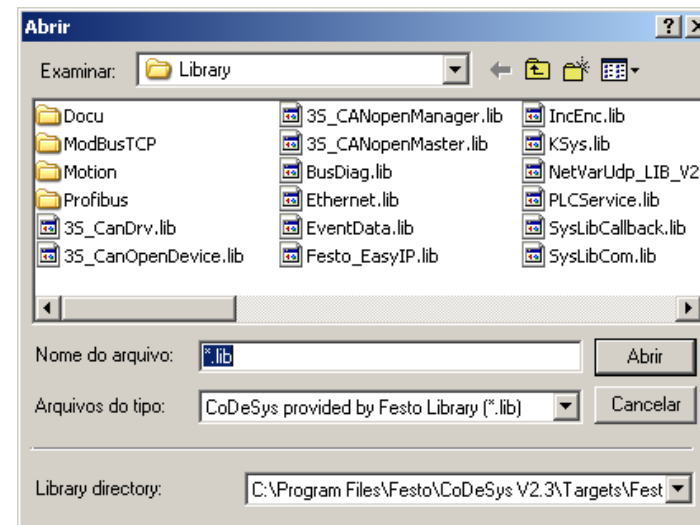
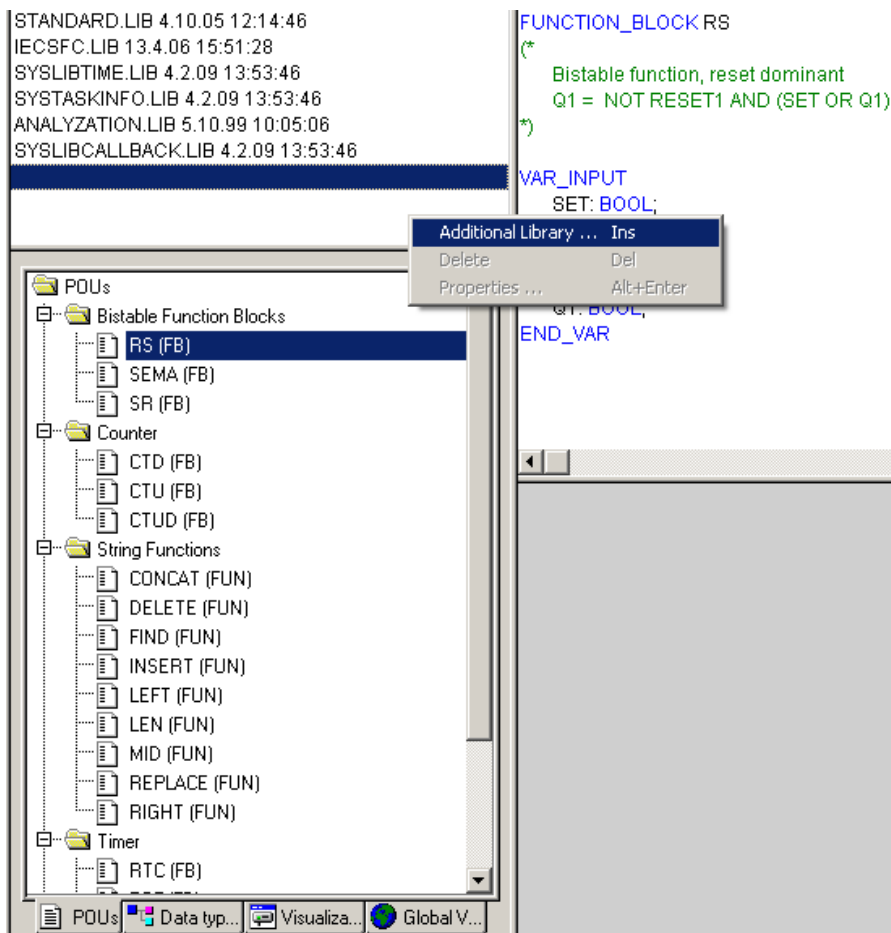
Formas de Endereçamento: Byte

%IX0.0- %IX0.7	%IX1.0- %IX1.7	%IX2.0- %IX2.7	%IX3.0- %IX3.7	%IX4.0- %IX4.7	%IX5.0- %IX5.7	%IX6.0- %IX6.7	%IX7.0- %IX7.7	%IX8.0- %IX8.7	%IX9.0- %IX9.7	
%IB0	%IB1	%IB2	%IB3	%IB4	%IB5	%IB6	%IB7	%IB8	%IB9	
	%IW1		%IW3		%IW5		%IW7		%IW9	
%IW0		%IW2		%IW4		%IW6		%IW8		
%ID0				%ID4				%ID8		
	%ID1				%ID5				%ID9	
Settings uration: Festo CECXX-C1 et PlatformMemory LayoutGeneralNetwork fun		%ID2				%ID6				
			%ID3				%ID7			



**Endereçamento por
Byte**

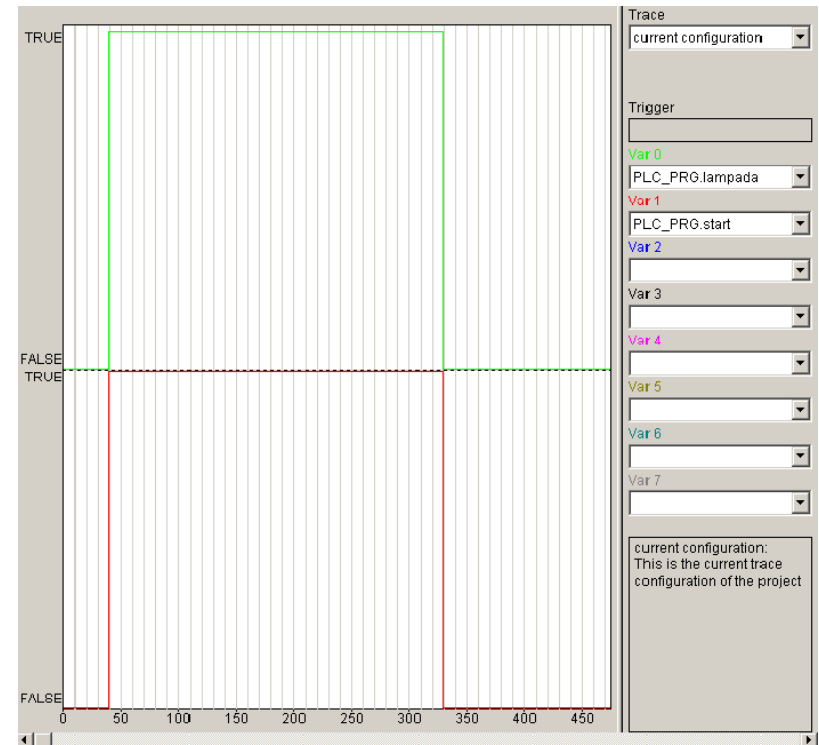
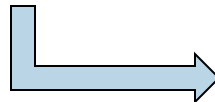
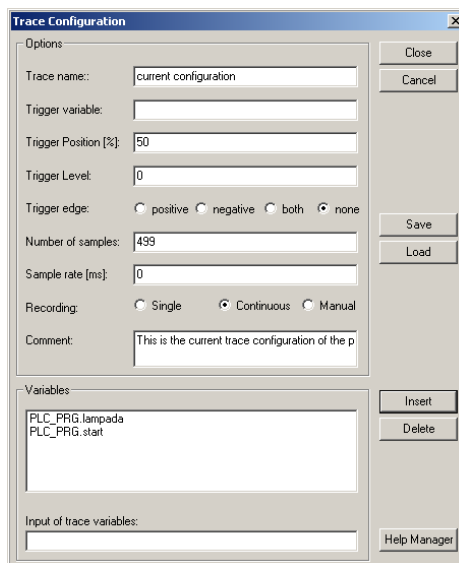
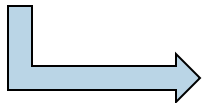
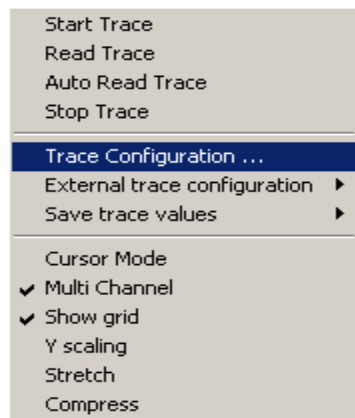
Resources – Library (Biblioteca)



É possível carregar bibliotecas adicionais.
A **extensão** do arquivo é **(.lib)**

Resources – Sampling Trace (amostragem de sinais)

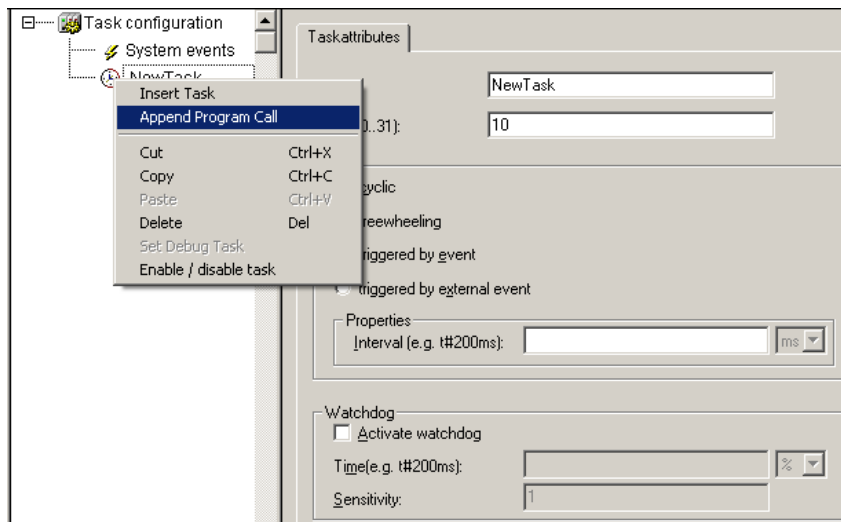
É um recurso que permite visualizar graficamente o comportamento de até 7 **variáveis** intrínseco ao programa. Para isso Clique com o botão direito do mouse e selecione **“Trace Configuration”**.



Para selecionar as variáveis no campo **“Input of trace variables”** pressione **Shift+F2** e selecione as variáveis do programa.

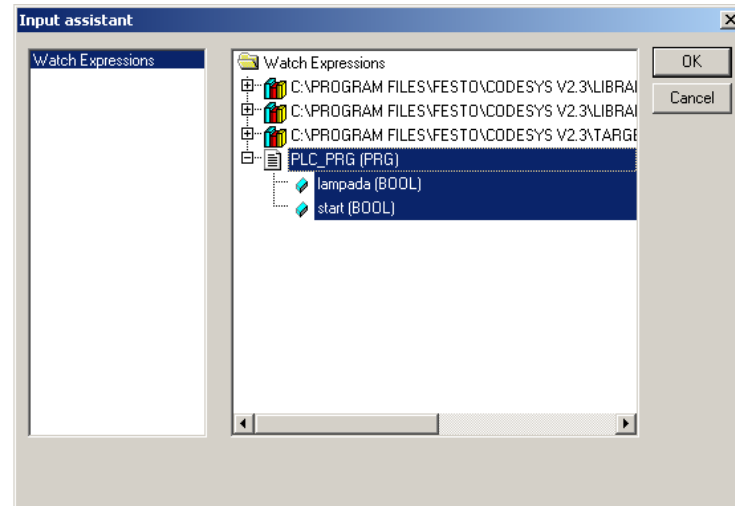
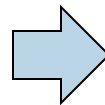
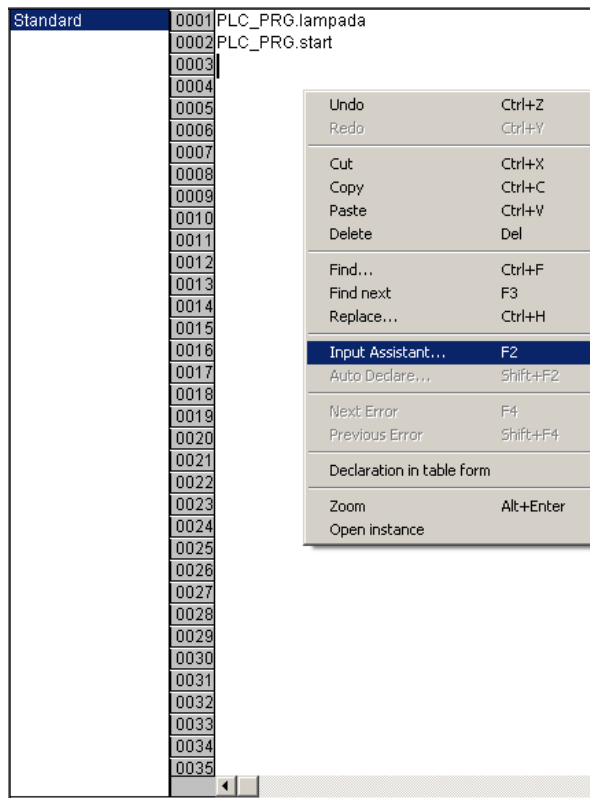
Resources – Task Configuration (configurações de tarefas)

Para inserir uma *task* (tarefa), basta clicar com o botão direito do mouse em **“System events”** e selecionar **“Insert Task”**. Com isso é possível realizar algumas atribuições como **prioridade**, **tipo de execução**, que o programa anexado a essa *task* execute durante sua operação. Para anexar o programa a *task*, basta clicar com o botão direito do mouse na mesma e selecionar **“Append Program Call”**.



Resources – Watch- and Recipe Manager

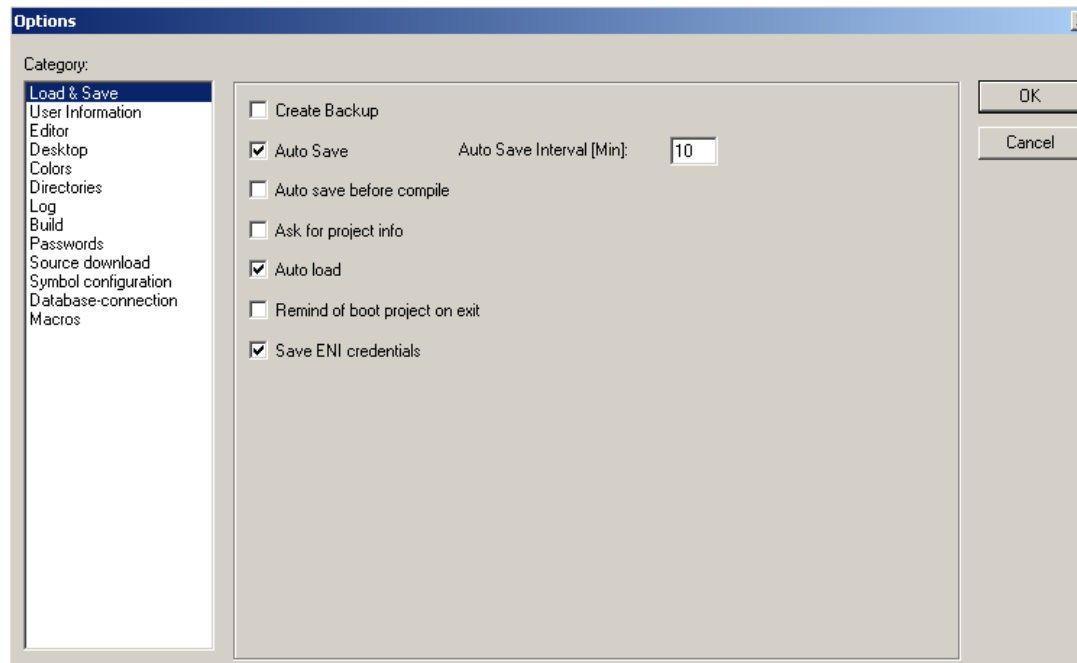
Este recurso serve como visualizador de estado das variáveis atrelados ao(s) programa(s).



Para isso, é necessário adicionar as variáveis. Clique com o botão direito do mouse e selecione **“Input Assistant”**, ou pressione **F2**. Na janela *Input Assistant* selecione as variáveis que deseja ser visualizadas.

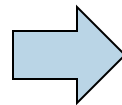
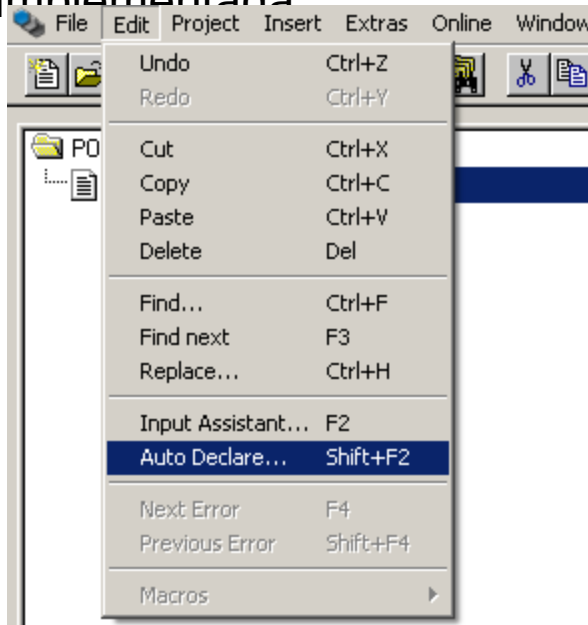
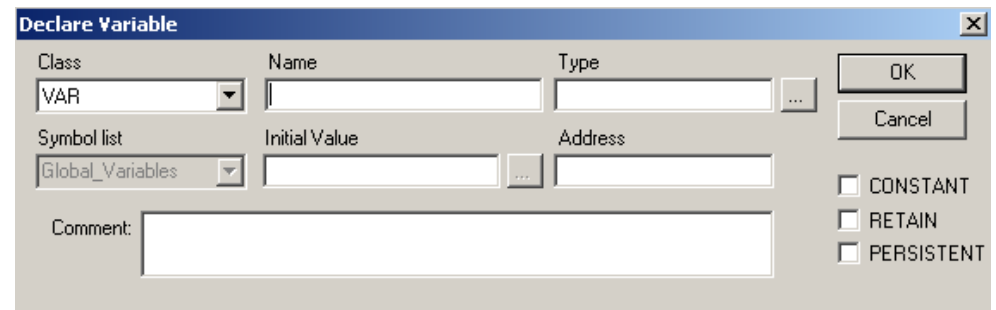
Resources – Workspace (Espaço de trabalho)

Este é um espaço que possui diversas opções de configuração de forma geral, independente do dispositivo que será utilizado. Neste é possível inserir senha de acesso ao projeto, adicionar informações de usuário, definir idioma, entre outras opções.



Declaração de Variáveis

Variáveis são elementos definidos de um programa que podem assumir diversos tipos de valores ou condições de funcionamento, variando conforme a lógica implementada.

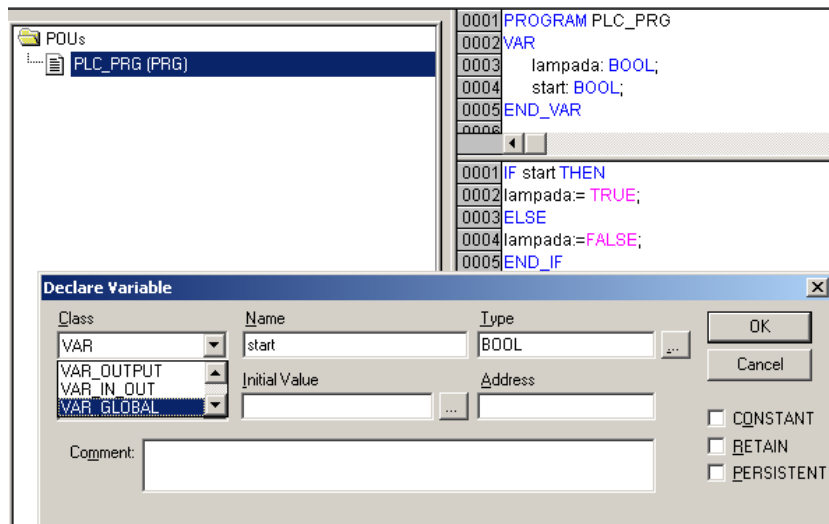
The image shows a screenshot of the 'Declare Variable' dialog box. The dialog box has a title bar with a close button. It contains several fields and options:

- Class:** A dropdown menu set to 'VAR'.
- Name:** An empty text field.
- Type:** An empty text field with a browse button (...).
- Symbol list:** A dropdown menu set to 'Global_Variables'.
- Initial Value:** An empty text field with a browse button (...).
- Address:** An empty text field with a browse button (...).
- Comment:** A large empty text area.
- Buttons:** 'OK' and 'Cancel' buttons.
- Options:** Three checkboxes labeled 'CONSTANT', 'RETAIN', and 'PERSISTENT', all of which are currently unchecked.

Resources – Global Variables (Variáveis Globais)

São variáveis que podem ser utilizados em todo o projeto. Quando declarada a mesma será reconhecida por todos os programas existentes no mesmo projeto. Para isso existe duas formas de declará-las.

1º Forma:

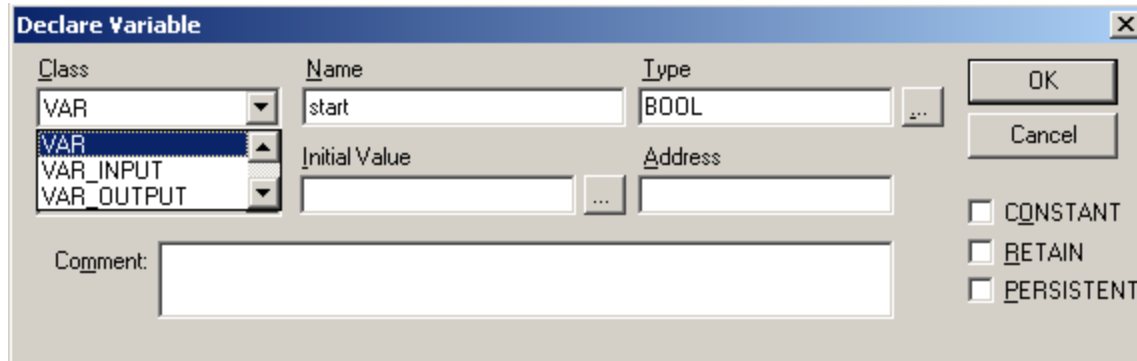


2º Forma:



Resources – Local Variables (Variáveis locais)

Devem ser declaradas no programa, e só serão reconhecidas pelo programa o qual foi declarada. É recomendado não utilizar variáveis absolutas (ex: %IX0.0), sem declarar uma variável local.



The image shows a 'Declare Variable' dialog box with the following fields and options:

- Class:** A dropdown menu with 'VAR' selected. Below it, a list shows 'VAR', 'VAR_INPUT', and 'VAR_OUTPUT'.
- Name:** A text field containing 'start'.
- Type:** A dropdown menu with 'BOOL' selected.
- Initial Value:** A text field with a small '...' button next to it.
- Address:** A text field with a small '...' button next to it.
- Comment:** A large text area.
- Buttons:** 'OK' and 'Cancel' buttons.
- Options:** Three checkboxes: 'CONSTANT', 'RETAIN', and 'PERSISTENT', all of which are currently unchecked.

Declaração de Variáveis

Classificação da variável:

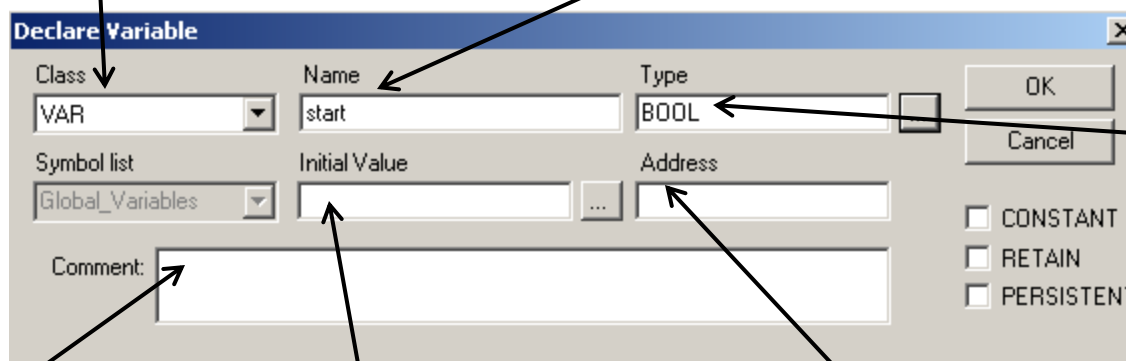
Local, Entrada, Saída, Global;

Nome:

Definido pelo programador;

Tipo de variável:

ARRAY
BOOL
BYTE
DATE
DINT
DT
DWORD
INT
LREAL
REAL
SINT
STRING
TIME
TOD
UDINT
UINT
USINT
WORD



The dialog box 'Declare Variable' contains the following fields and options:

- Class:** A dropdown menu currently showing 'VAR'.
- Name:** A text input field containing 'start'.
- Type:** A dropdown menu currently showing 'BOOL'.
- Symbol list:** A dropdown menu currently showing 'Global_Variables'.
- Initial Value:** A text input field with a button to its right.
- Address:** A text input field with a button to its left.
- Comment:** A large text area at the bottom.
- Buttons:** 'OK' and 'Cancel' buttons on the right.
- Options:** Three checkboxes labeled 'CONSTANT', 'RETAIN', and 'PERSISTENT'.

**Comentário
s:**

Opcional;

**Atribuição
de valor
inicial:**

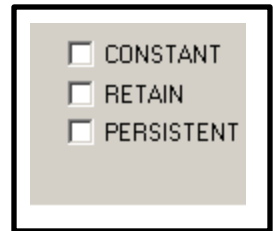
Opcional;

Endereço:

Pode ser atribuído um
endereço Absoluto;

Declaração de Variáveis

As memórias possuem 3 tipos de configuração: **Var**, **Var Persistent**, **Var Retain** e **Var Persistent Retain**.



- **Var** : Este tipo ao desligar o PLC ela perderá seu valor voltando ao seu estado de inicialização.
- **Var Persistent**: Ao efetuar o download as variáveis deste tipo mantém seu ultimo valor.
- **Var Retain** : Ao desligar o equipamento estas variáveis não perdem o seu ultimo valor.
- **Var Persistent Retain**: Ao desligar ou fazer o download ela não perde seu ultimo valor.

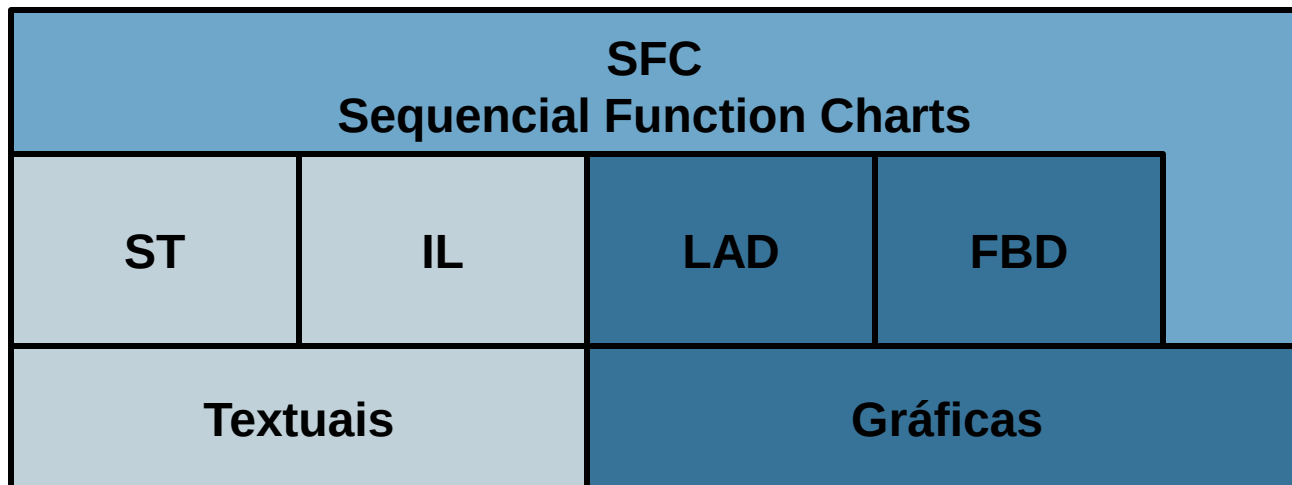
Norma IEC 61131

- Tem como objetivo unificar as características de Software e Hardware para Controladores Programáveis.

Parte	Título
1	Informações Gerais
2	Requisitos e testes do equipamento (Hardware)
3	Linguagens de Programação
4	Guia do usuário
5	Comunicação
6	Reservada
7	Programação para Controle Fuzzy
8	Orientação para implementação de linguagens de programação

Linguagens de Programação

- Padronização de 5 linguagens de programação:



Linguagens de Programação

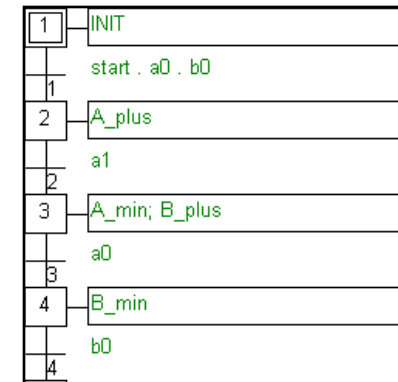
IL (Instruction List)

```
LD Push-button_a
AND Push-button_b
OR(Push-button_b
AND Push-button_c
)
OR(Push-button_c
AND Push-button_a
)
ST Press_P
```

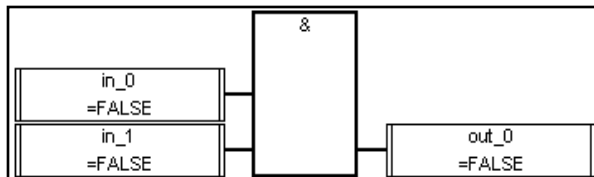
ST (Structured Text)

```
A_plus := start AND a0 AND b0;
A_min := b1;
B_plus := a1 AND b0;
B_min := b1;
```

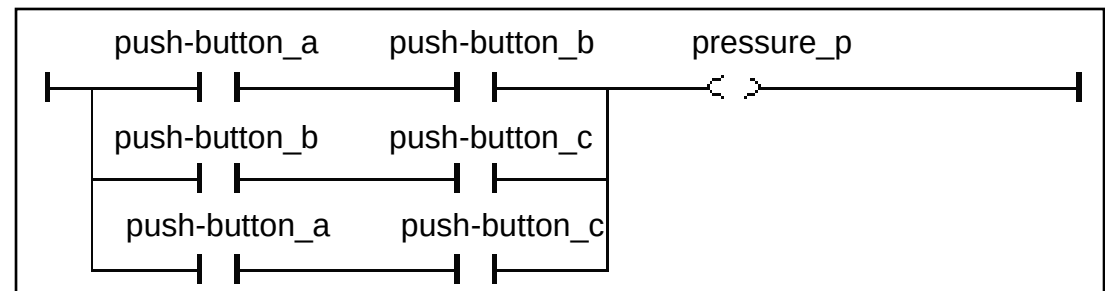
SFC (Sequential Function Chart)



FBD (Function Block Diagram)

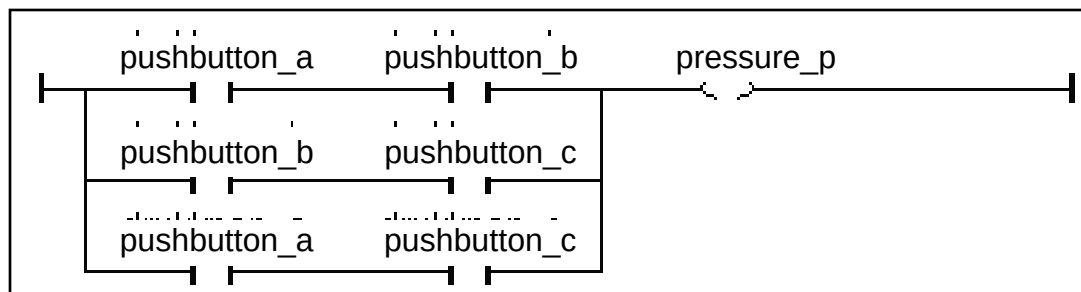


LD (Ladder Diagram)



IEC 61131-3: LD (Ladder Diagram)

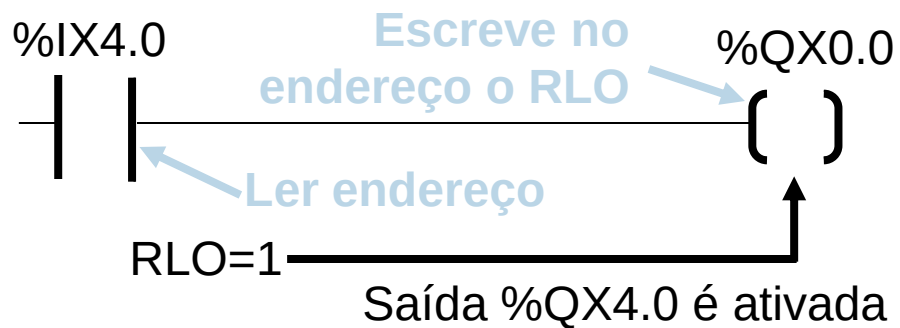
- Linguagem de programação baseada em diagrama de contatos elétricos;
- Adequada para controle discreto, combinacional e seqüenciamento (intertravamento);
- Utiliza blocos de função para controle e funções especiais.



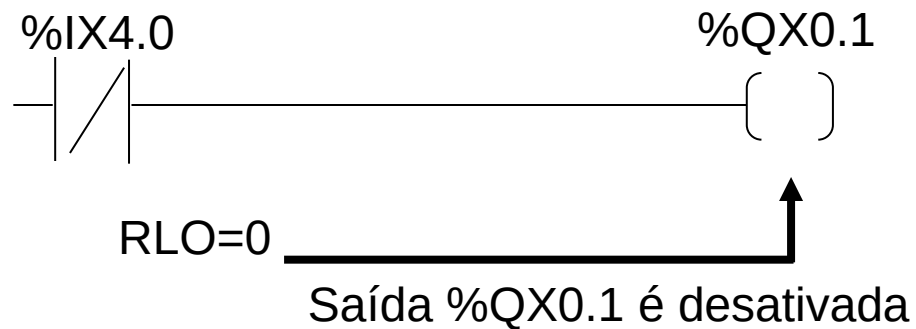
Instruções em Diagrama Ladder

Exemplo: Entrada %IX4.0 é verdadeira

Instrução:
Verifica se endereço %IX4.0 é verdadeiro

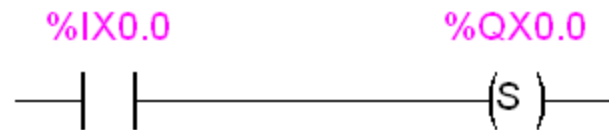


Instrução:
Verifica se endereço %IX4.0 é falso

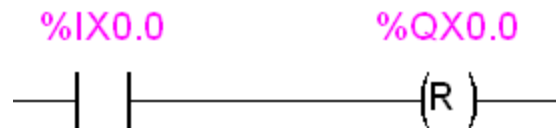


Instruções Diagrama Ladder: Set e Reset

- **Set:** Liga um bit e o mantém ativo;



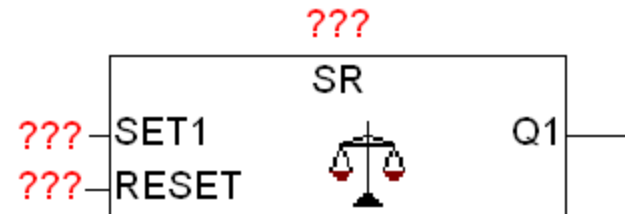
- **Reset:** Desliga um bit e o mantém desativado;



Instruções em Linguagem Ladder – Função Set/Reset

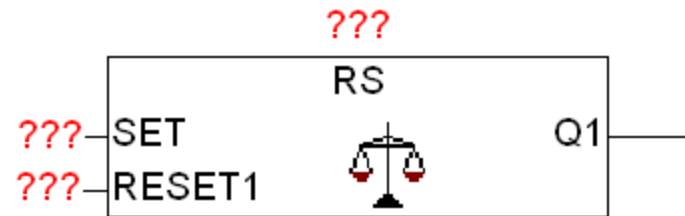
- SR:

SET 1	RESE T	Q1
0	0	Mantém
0	1	0
1	0	1
1	1	1



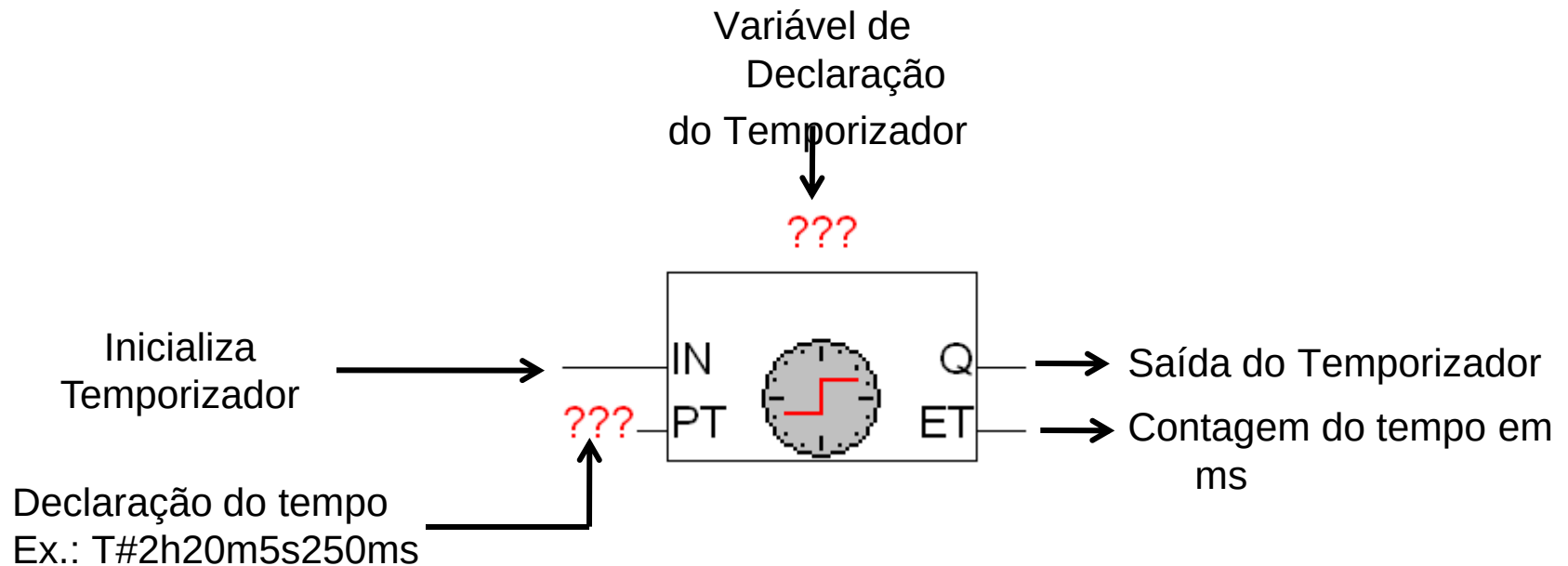
- RS:

SE T	RESET 1	Q1
0	0	Mantém
0	1	0
1	0	1
1	1	0

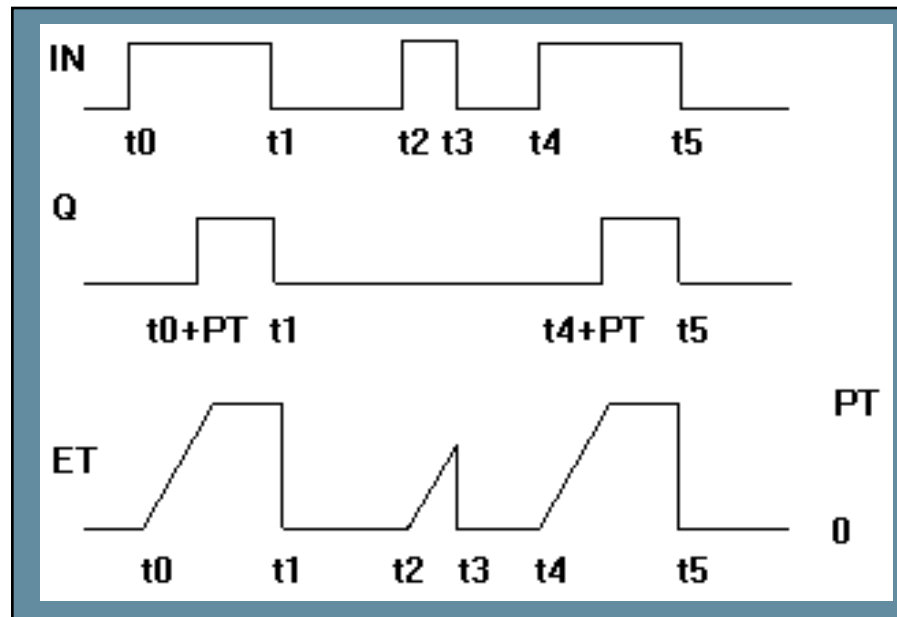
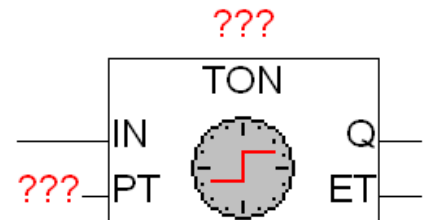


Temporizadores

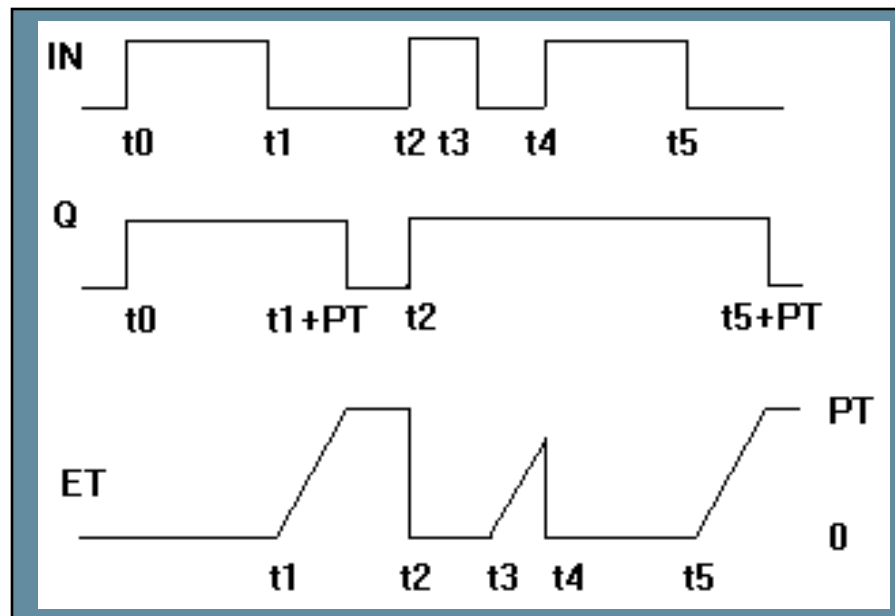
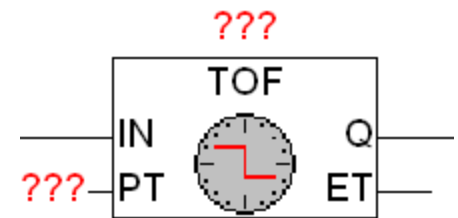
- 3 tipos de temporizadores (TON, TOF e TP)
- Bloco do Temporizador em Diagrama Ladder



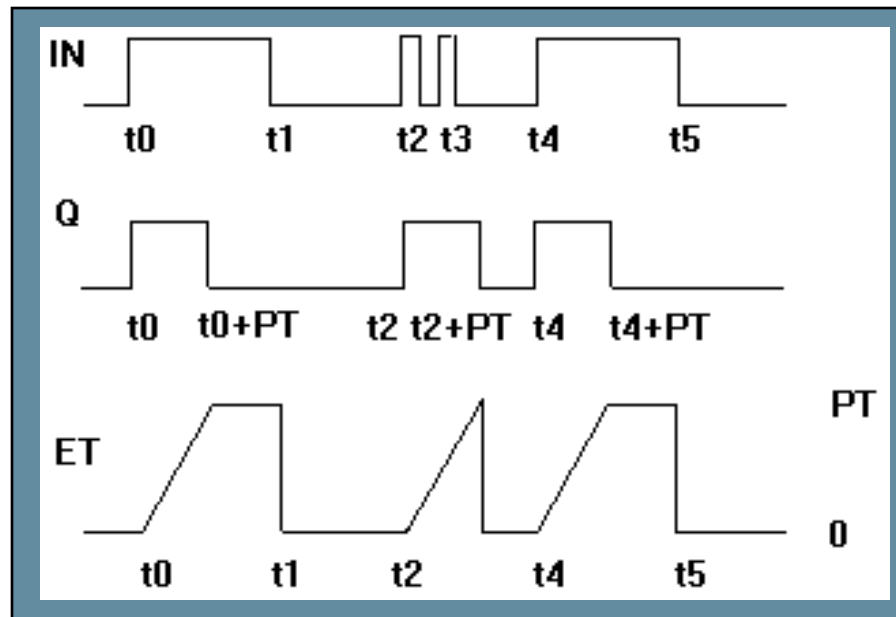
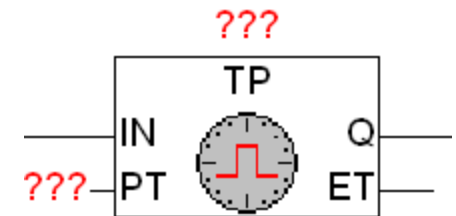
Temporizador com Retardo na Ativação (TON)



Temporizador com Retardo na Desativação (TOF)

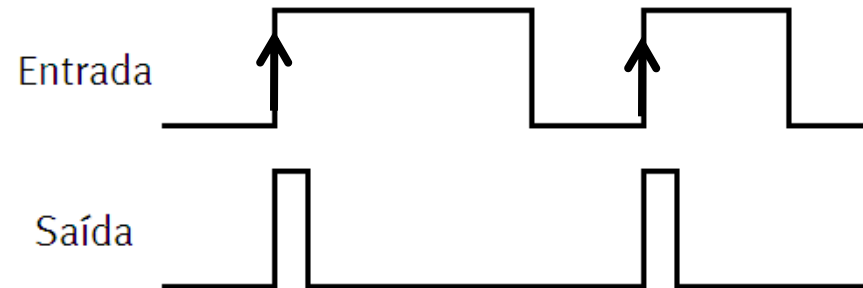
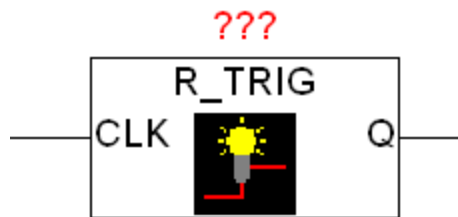


Temporizador por Pulso (TP)

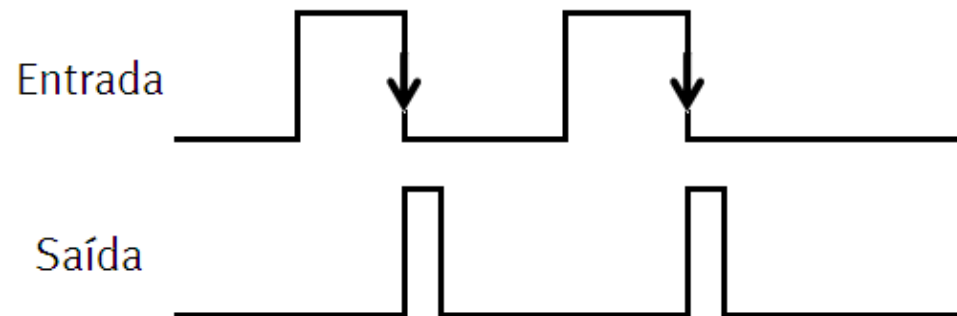
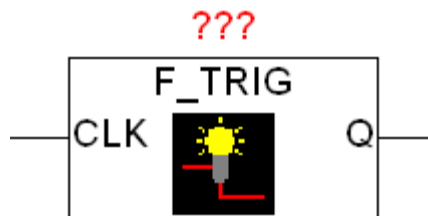


Detecção de Bordas

- Borda de Subida



- Borda de Descida

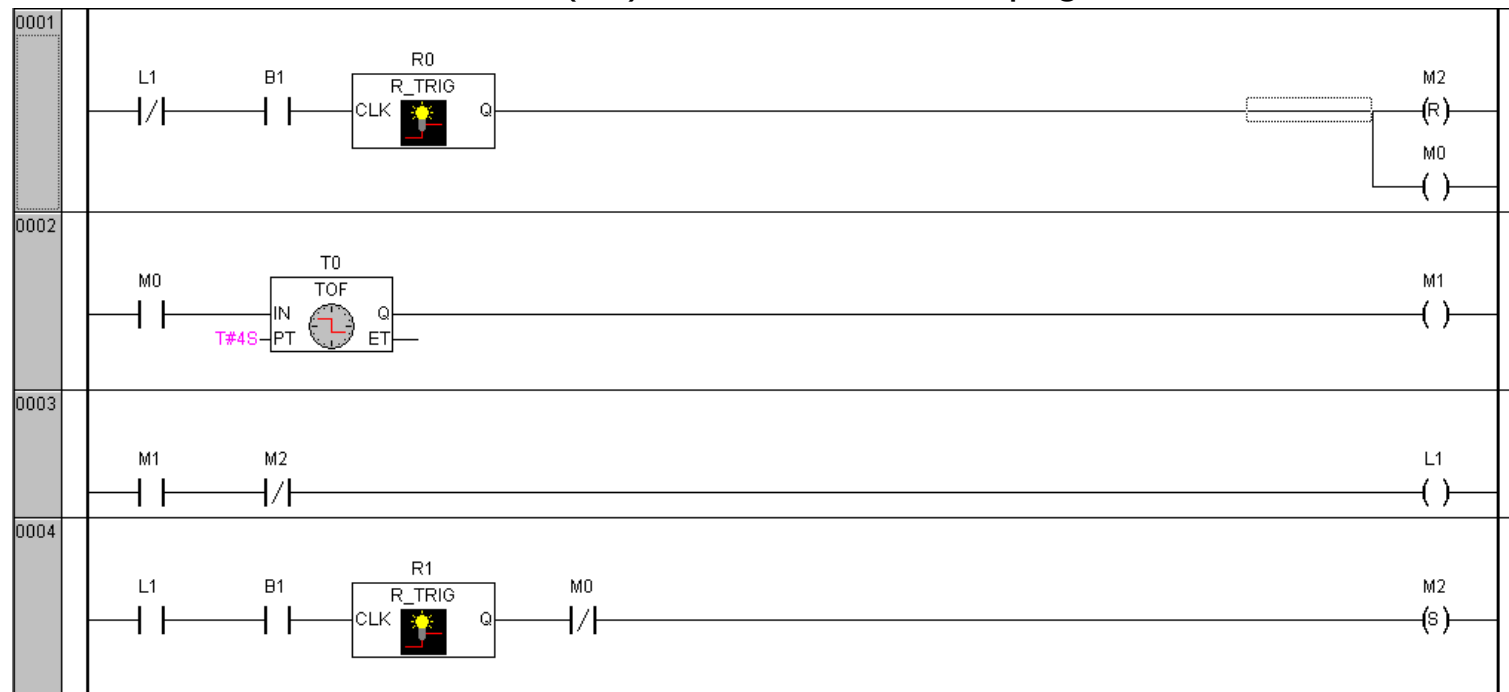


Exercício proposto 01 (Ladder)

Desenvolver um programa que ao pressionar o **botão(B1)**, acenda a **lâmpada (L1)**, onde a mesma deve permanecer acesa mediante as seguintes condições:

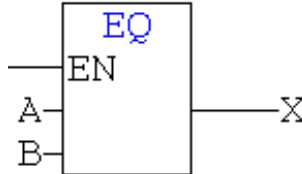
1º Após 4 segundos L1 deverá apagar; ou

2º Ao pressionar novamente o botão(B1) a mesma deverá apagar;



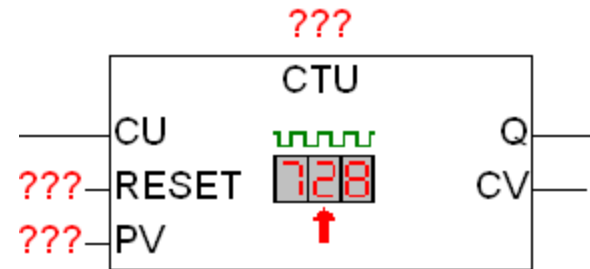
Funções de Comparação

- Comparar duas variáveis de entrada, A e B. (Saída é Verdadeira ou Falsa)

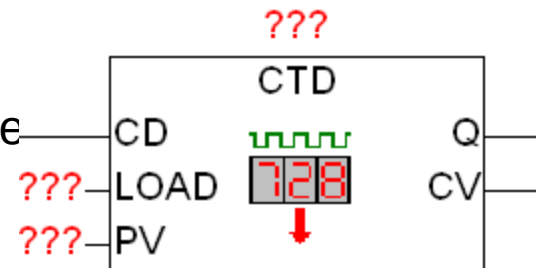
Operator	LD	Descrição
EQ		A igual a B
NE		A diferente de B
GE		A maior ou igual a B
GT		A maior que B
LE		A menor ou igual a B
LT		A menor que B

Contadores

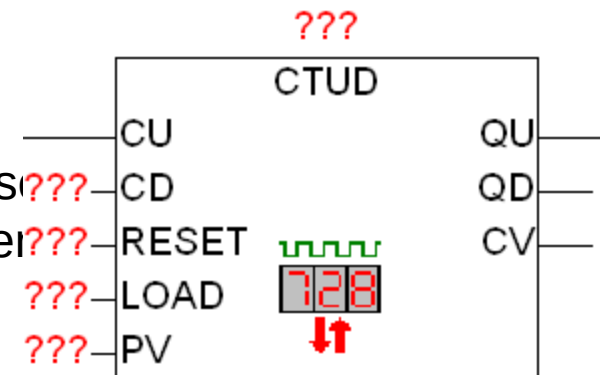
- **CTU** Contador crescente



- **CTD** Contador Decrescente



- **CTUD** Contador Cres
Decrescente dependente
da entrada

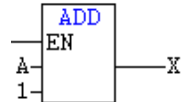
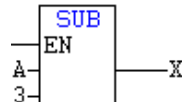
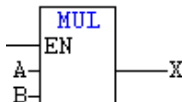
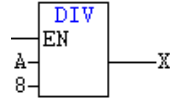
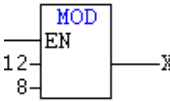


Exercício proposto 02 (Ladder)

Deseja-se desenvolver um sistema que contará a quantidade de peças que passam por uma esteira. Para isso o sensor detector de peças (S1) irá acionar toda vez que uma peça for detectada. O sistema será ligado (a esteira entrará em funcionamento) quando a chave retentiva (CH0) for posicionada em ON, sendo que a lâmpada (L0) deverá acender indicando este funcionamento. Para isso deverá ser considerado pelo projetista (você) o seguinte:

- 1º Ao detectar 5 peças a Lâmpada L1 deverá acender;
- 2º Ao detectar 7 peças a lâmpada L1 deverá apagar e acender a lâmpada L2;
- 3º Ao detectar 10 peças, a lâmpada L2 deverá apagar e acender a lâmpada L3. Neste instante a esteira deverá parar, onde só entrará em funcionamento novamente mediante o reset dado por um botão pulsador (B1) ligado ao contador.
- 4º Quando pressionado o botão de reset (B1) as Lâmpadas L1 L2 e L3 deverão apagar, iniciando novamente o processo de contagem.

Funções Aritméticas

Operador	LD	Função
ADD		$X = A + 1;$
SUB		$X = A - 3;$
MUL		$X = A * B;$
DIV		$X = A / 8;$
MOD		$X = 12 \text{ MOD } 8;$ $(R = 4)$

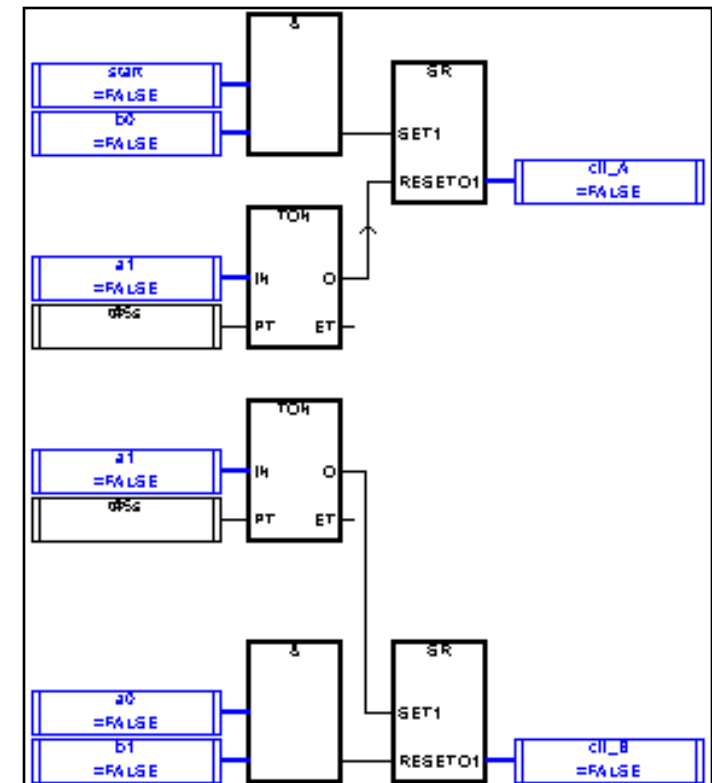
IEC 61131-3: IL (Instruction List)

- Linguagem de Baixo Nível semelhante ao Assembly;
- Recomendada para pequenas aplicações ou otimização de códigos;
- Linguagem básica para exportação de programas (Portabilidade);

```
LD Push-button_a  
AND Push-button_b  
OR(Push-button_b  
AND Push-button_c  
)  
OR(Push-button_c  
AND Push-button_a  
)  
ST Press_P
```

IEC 61131-3: FBD (Function Block Diagram)

- Baseada nos diagramas de circuitos (Diagrama Lógico);
- Adequada para controle discreto, seqüencial, regulatório e etc;
- Representação de fácil interpretação;
- Blocos expansíveis em função do número de parâmetros de entrada;
- Blocos encapsulam o algoritmo, destacando o fluxo informações e o processamento de sinais.



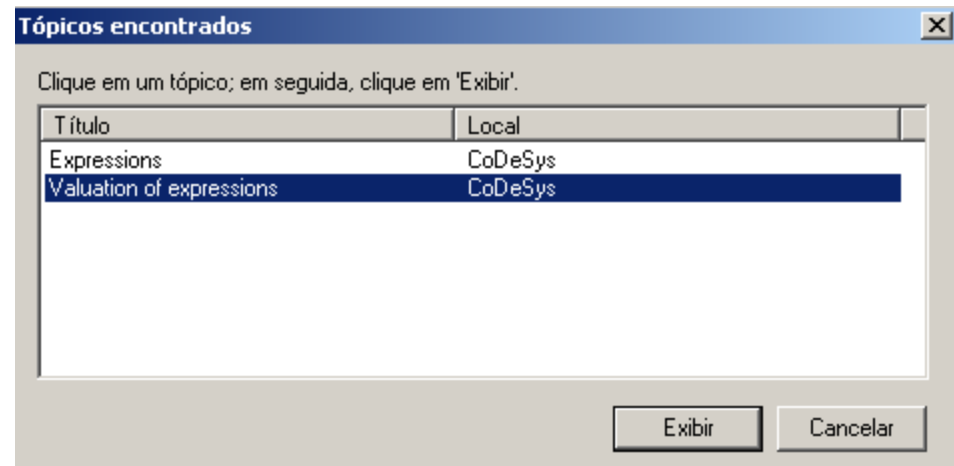
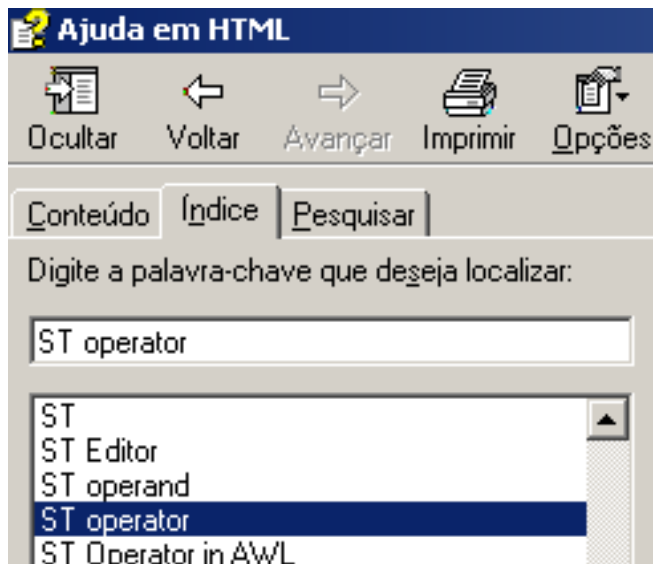
IEC 61131-3: ST (Structured text)

- Linguagem de programação de alto nível, semelhante ao Pascal;
- Ideal para tomada de decisões, cálculos, criação de blocos funcionais;
- Suporte a funções tais como:
 - Loops (REPEAT-UNTIL; WHILE-DO)
 - Condições (IF-THEN-ELSE; CASE)
 - Funções matemáticas (SQRT(), SIN())

```
A_plus := start AND a0 AND b0;  
A_min := b1;  
B_plus := a1 AND b0;  
B_min := b1;
```

IEC 61131-3: ST (Structured text)

Através do “**Help (F1)**” é possível verificar os **tipos de instruções**, com os comandos que devem ser realizados para cada situação, e as possíveis **operações** que podem ser realizadas ao longo do programa.



Exercicio proposto 03 (Structured Text)

Desenvolver um programa que ao pressionar o **botão(B1)**, acenda a **lâmpada (L1)**, onde a mesma deve permanecer acesa mediante as seguintes condições:

1º Após 4 segundos L1 deverá apagar; ou

2º Ao pressionar novamente o botão(B1) a mesma deverá apagar;

```
PROGRAM BOTAO
VAR
  L1: BOOL;
  M0: INT;
  TIMER: TOF;
  T0: TOF;
  M1: BOOL;
  B1: BOOL;
END_VAR
```

```
T0.IN:=B1;
T0.PT:=T#4S;
M1:=T0.Q;
  T0;

      CASE M0 OF

0:    IF B1 THEN
      L1:=TRUE;
      M0:=5;
      END_IF

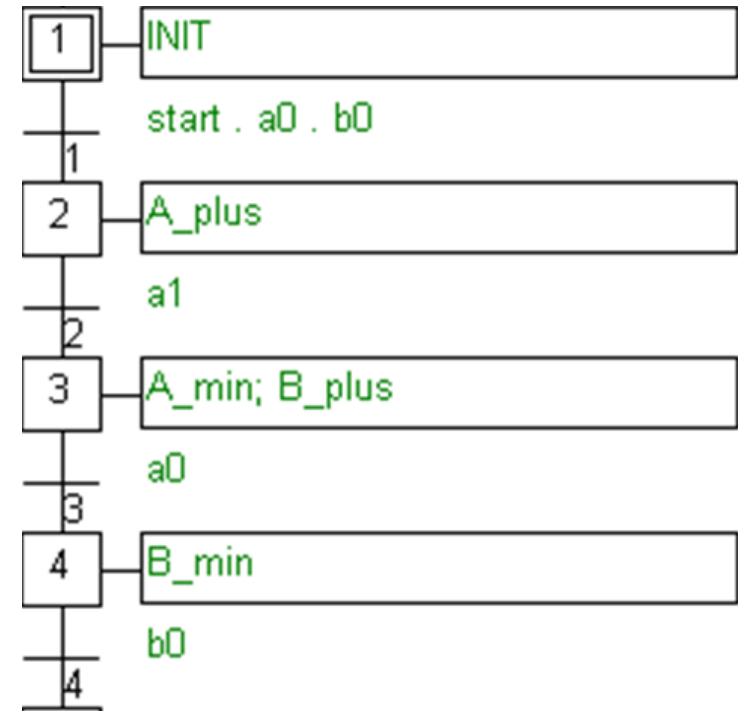
5:    IF NOT B1 THEN
      M0:=10;
      END_IF

10:   IF B1 OR T0.ET=T#4S THEN
      L1:=FALSE;
      M0:=20;
      END_IF

20:   IF NOT B1 THEN
      M0:=0;
      END_IF
      END_CASE
```

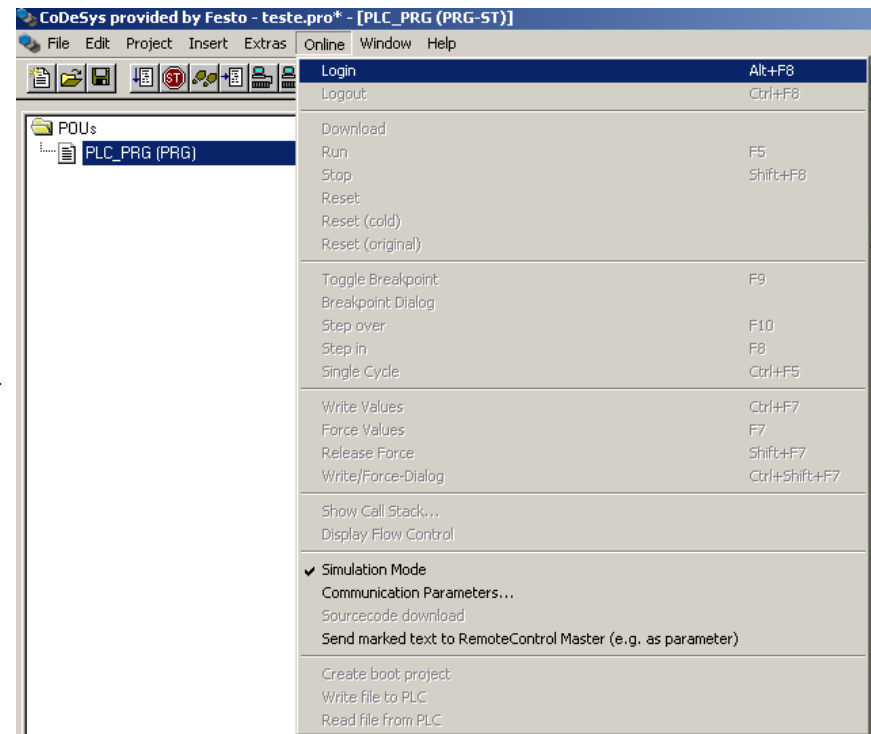
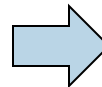
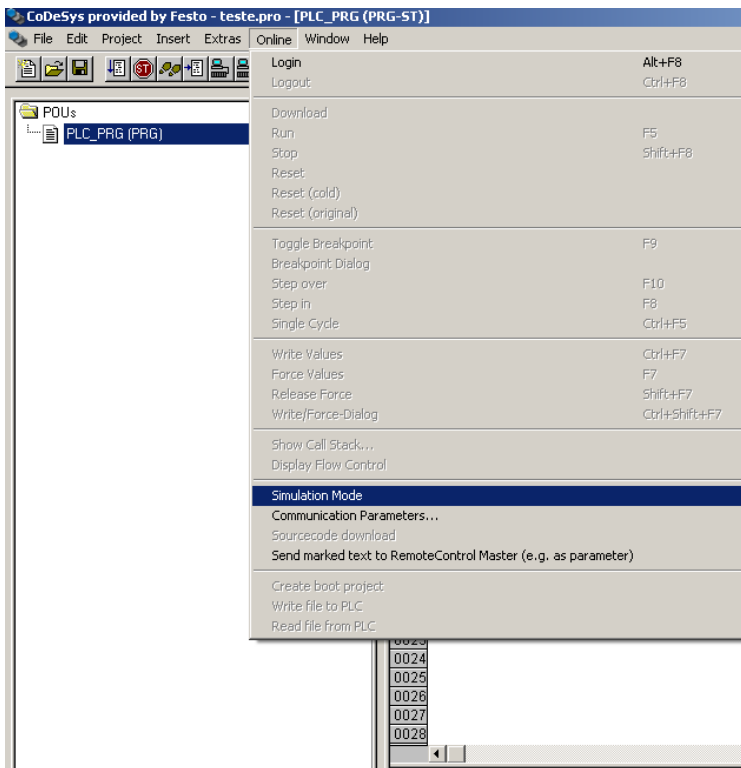
IEC 61131-3: SFC (Sequential Function Chart)

- Descreve graficamente o comportamento seqüencial de um programa de controle;
- É a única linguagem gráfica da norma que também pode ser programada de forma textual;
- Linguagem adotada para programação de sistemas de bateladas (batchs);
- Fácil detecção, diagnóstico, de ocorrência de erros;



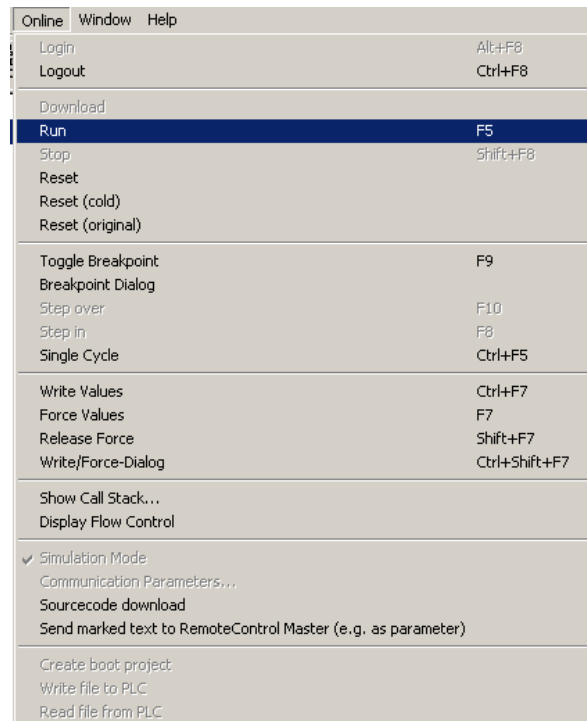
Simulation Mode (Modo simulação)

Neste modo, o programa pode ser simulado, possibilitando a verificação e a manipulação das variáveis atreladas ao programa.



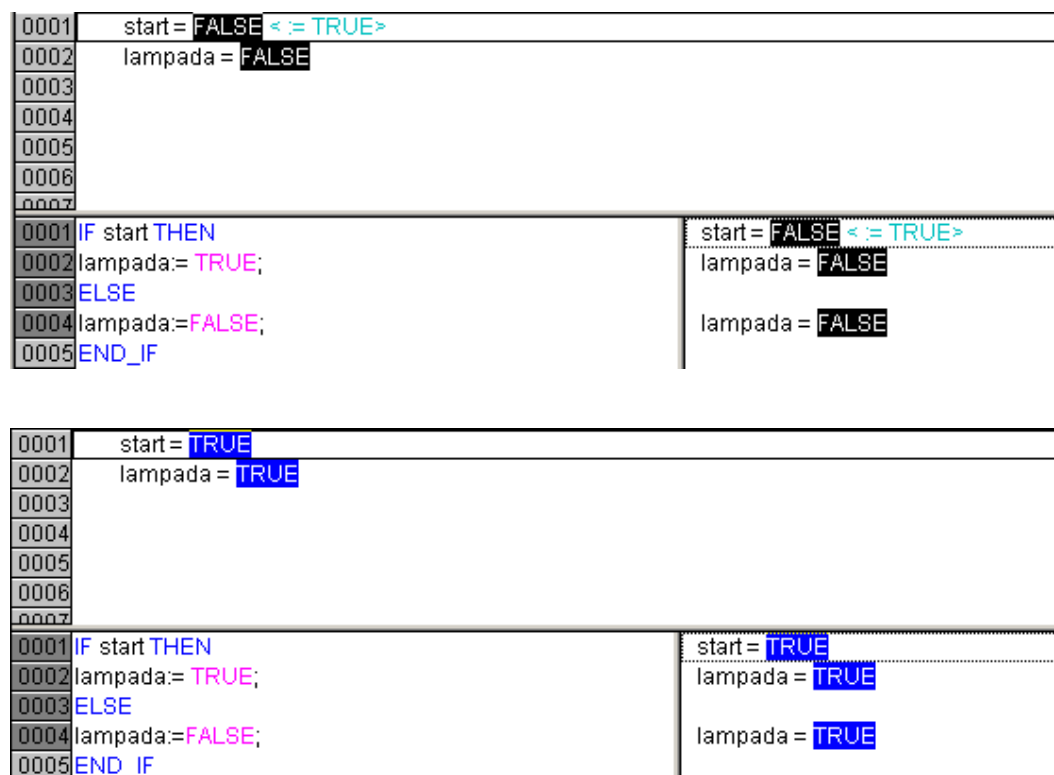
Simulation Mode (Modo simulação)

Quando selecionado o item **login** o programa é compilado automaticamente. Caso não exista erro, o programa pode ser colocado em modo **run**, possibilitando a visualização em tempo real das variáveis programadas.



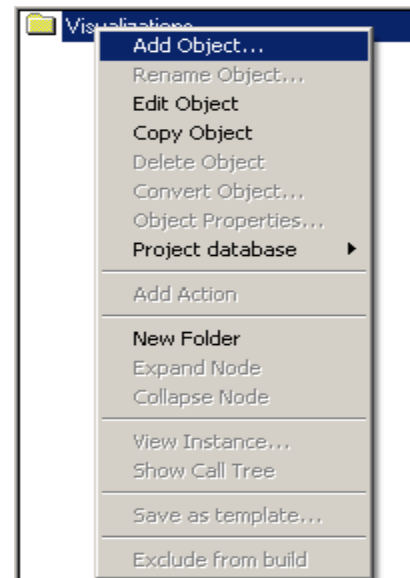
Simulation Mode (Modo simulação)

As variáveis de entrada podem ser manipuladas dando um **duplo clique**.
Pressionando **Ctrl + F7** altera-se a condição de estado da variável.



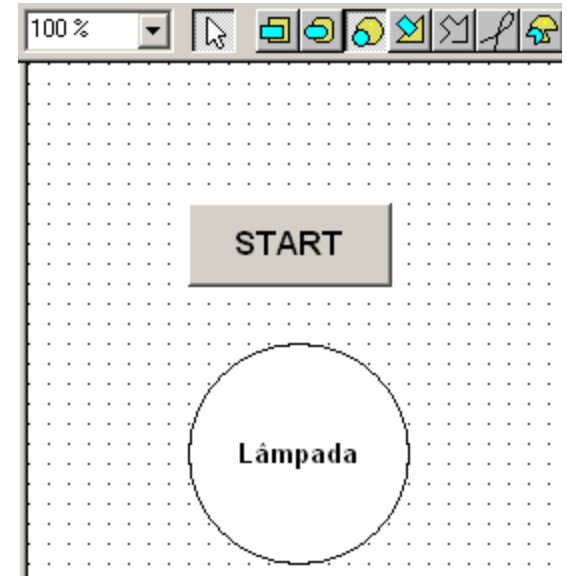
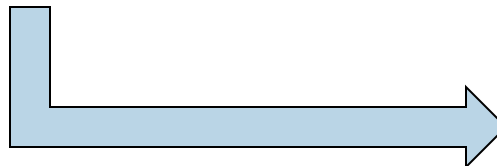
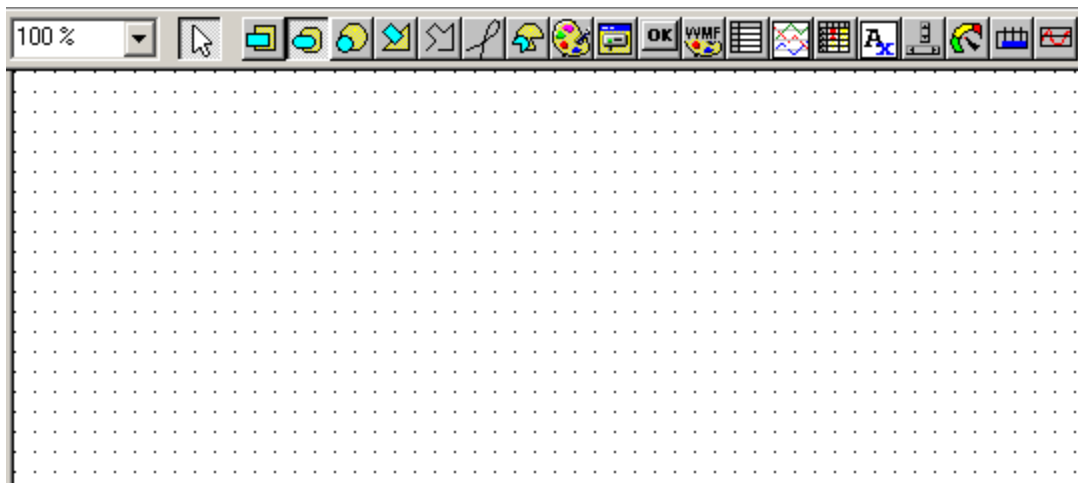
Visualizations (Visualizações)

Esta é uma ferramenta que permite desenvolver telas para visualizações gráficas do comportamento das variáveis do programa. Uma das vantagens dessa ferramenta é simular um sistema real sem que se tenha o controlador operando, ou desenvolver uma IHM . Para isso deve-se selecionar a aba **visualization** e, clicando com o botão direito do mouse selecionar **Add Object**.



Visualizations (Visualizações)

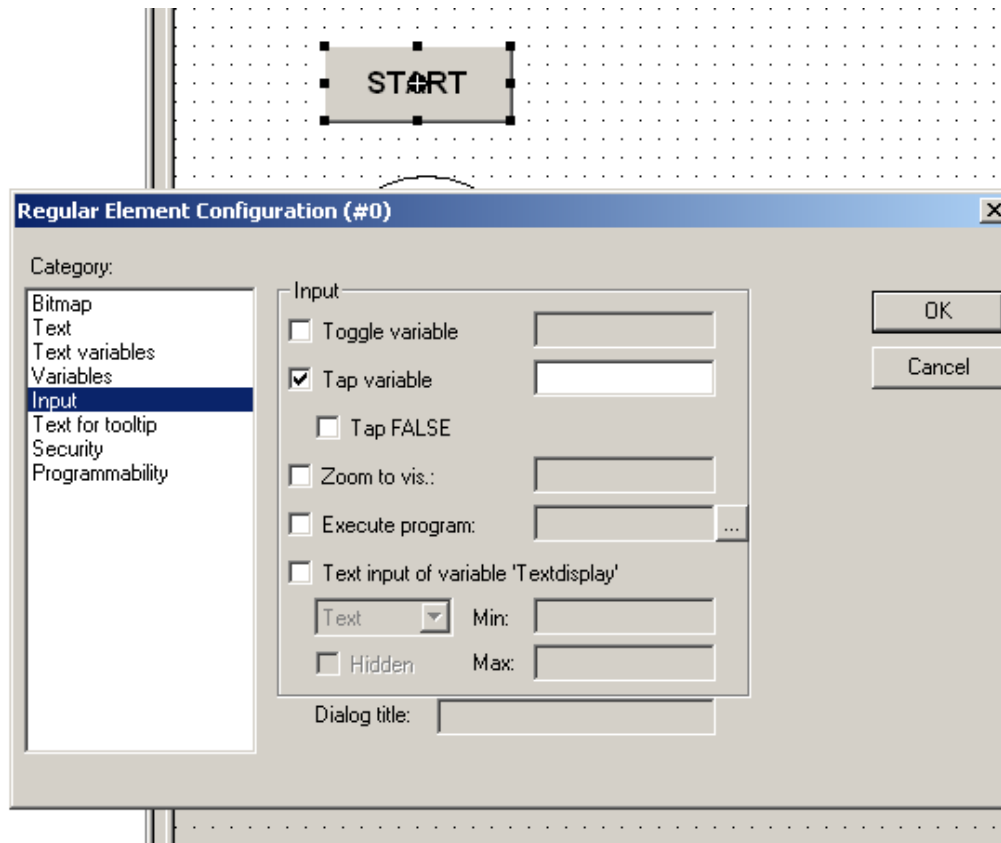
Através da barra de ferramentas é possível desenhar os componentes desejados.



Visualizations (Visualizações)

Para criar um link entre o botão desenhado e a variável do programa, é necessário:

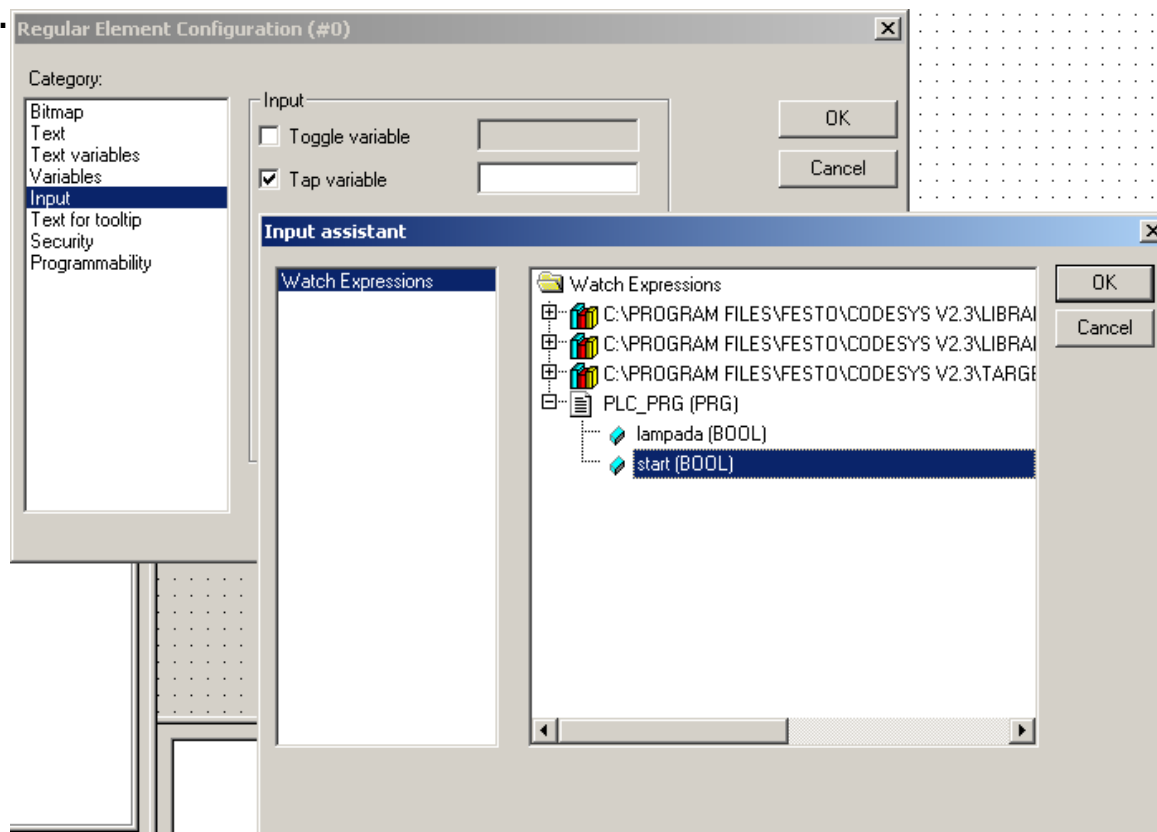
1º passo:



Visualizations (Visualizações)

No “campo” **Tap Variable** selecione o botão **F2** e selecione a variável que deseja atrelar ao botão.

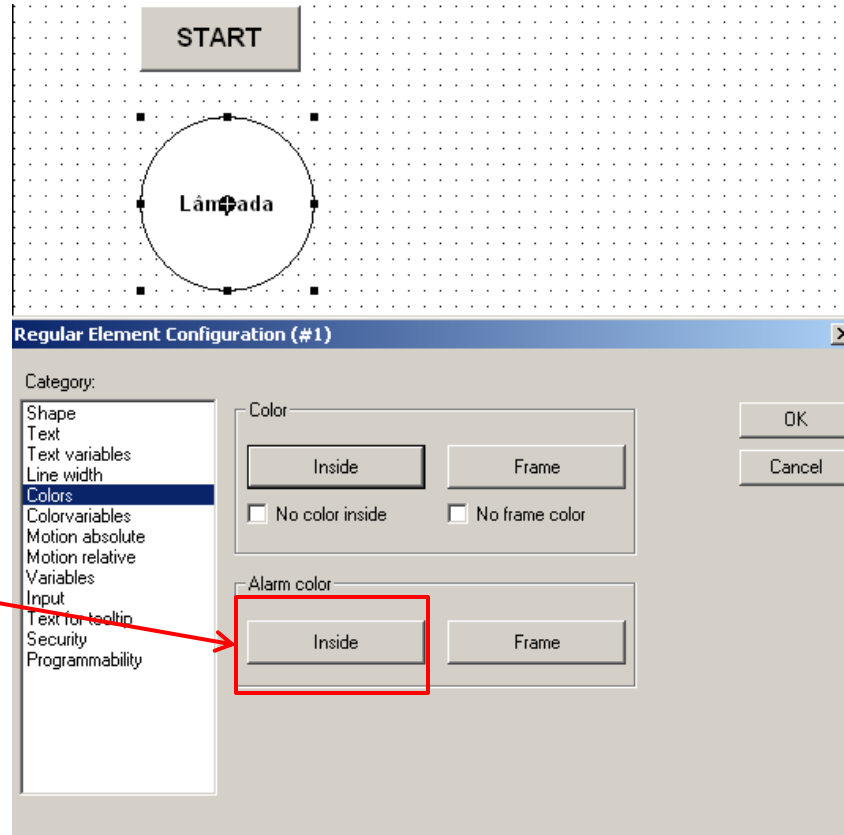
2º passo:



Visualizations (Visualizações)

Para a lâmpada é possível alterar a cor conforme seu funcionamento. Portanto:

3º passo:

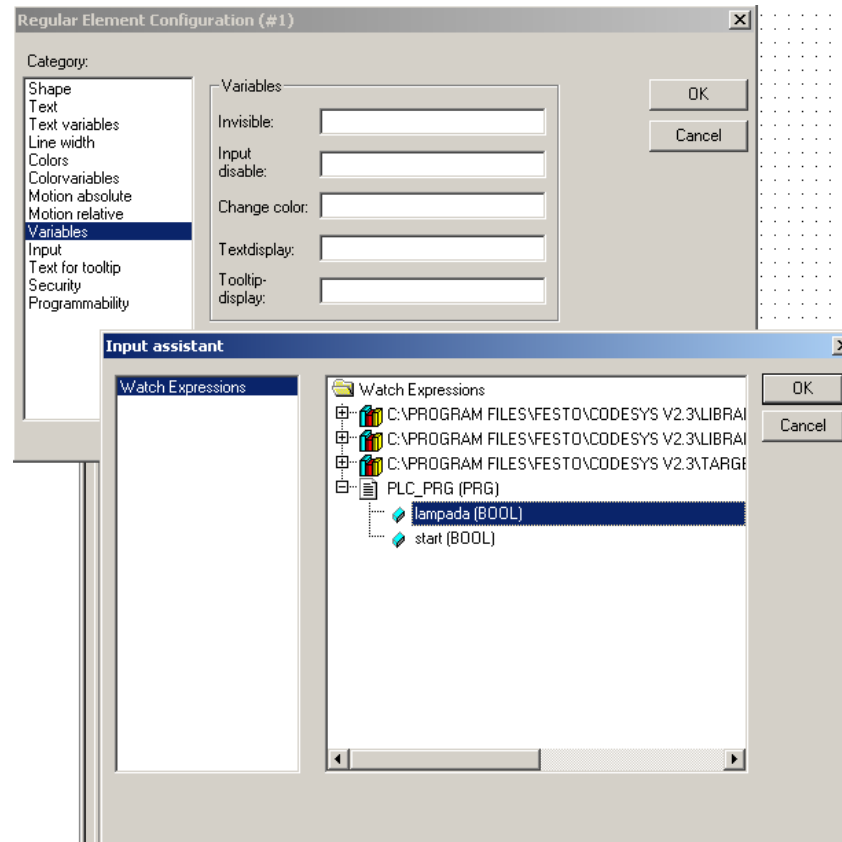


Selecionar a cor que o
desenho lâmpada irá
assumir quando
acionado

Visualizations (Visualizações)

Pressionado **F2** no campo **change color** e selecione a variável desejada.

4º passo:



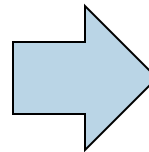
Visualizations (Visualizações)

Em **run**, verifique a visualização resultante. Quando pressionado **START**, a Lâmpada acende.

```

PLC_PRG (PRG-ST)
0001 start = FALSE
0002 lampada = FALSE
0003
0004
0005
0006
0007
0001 IF start THEN
0002 lampada:= TRUE;
0003 ELSE
0004 lampada:=FALSE;
0005 END_IF
  
```

Teste



```

PLC_PRG (PRG-ST)
0001 start = TRUE
0002 lampada = TRUE
0003
0004
0005
0006
0007
0001 IF start THEN
0002 lampada:= TRUE;
0003 ELSE
0004 lampada:=FALSE;
0005 END_IF
  
```

Teste

START



START

